

BÁO CÁO ĐỒ ÁN / TIỂU LUẬN

DỰ ÁN E-COMMERCE MICROSERVICE

Phân tích use-case, thiết kế hệ thống, hiện thực mã nguồn, AI service và kết quả triển khai

Họ và tên: Mai Thế Dương

Lớp: E22CNPM01

Mã sinh viên: B22DCDT069

Đề tài: E-commerce Microservice

Ngày 11 tháng 6 năm 2026

Tóm tắt nội dung

Báo cáo này trình bày dự án **e-commerce-microservice** theo hướng từ nền tảng Monolithic, Microservices và DDD đến phân tích, thiết kế, hiện thực và triển khai hệ thống e-commerce hoàn chỉnh. Nội dung bao quát các service nghiệp vụ chính, AI service, cơ chế giao tiếp giữa các thành phần, cấu hình Docker và luồng mua hàng end-to-end. Những phần chưa tách thành service độc lập như Cart, Shipping, JWT hay Nginx được mô tả ở mức logic hiện tại và định hướng mở rộng.

Mục lục

1	CHƯƠNG 1: TỪ MONOLITHIC ĐẾN MICROSERVICES VÀ DDD	5
1.1	Giới thiệu Monolithic Architecture	5
1.1.1	Khái niệm	5
1.1.2	Cấu trúc điển hình	5
1.1.3	Ví dụ thực tế	5
1.1.4	Nhược điểm chi tiết	5
1.1.5	Khi nào nên dùng Monolithic	6
1.2	Microservices Architecture	6
1.2.1	Khái niệm	6
1.2.2	Đặc điểm cốt lõi	6
1.2.3	So sánh Monolithic và Microservices	6
1.2.4	Ưu điểm của Microservices	7
1.2.5	Nhược điểm của Microservices	7
1.2.6	Nguyên tắc thiết kế	8
1.3	Domain Driven Design (DDD)	8
1.3.1	Mục tiêu	8
1.3.2	Các khái niệm cốt lõi	8
1.3.3	Context Map	8
1.3.4	DDD trong Microservices	9
1.4	Case Study: Hệ thống Quản lý Bệnh viện (Healthcare)	9
1.4.1	Mô tả bài toán	9
1.4.2	Quy trình phân rã theo DDD	9
1.4.3	Ví dụ API	10
1.5	Kết luận Chương 1	10
2	CHƯƠNG 2: PHÁT TRIỂN HỆ THỐNG E-COMMERCE MICROSERVICES	11
2.1	Xác định yêu cầu hệ thống	11
2.1.1	Yêu cầu chức năng (Functional Requirements)	11
2.1.2	Yêu cầu phi chức năng (Non-functional Requirements)	11
2.2	Phân rã hệ thống theo DDD	12

2.2.1	Bounded Context và ánh xạ Service	12
2.2.2	Nguyên tắc thiết kế	12
2.3	Biểu đồ Use Case nghiệp vụ	13
2.4	Thiết kế Product Service	15
2.4.1	Biểu đồ lớp (Class Diagram)	15
2.4.2	Data Model (sinh từ Class Diagram)	15
2.4.3	API	16
2.5	Thiết kế User Service	16
2.5.1	Biểu đồ lớp (Class Diagram)	16
2.5.2	Data Model	16
2.5.3	Phân quyền RBAC	17
2.5.4	API	17
2.6	Thiết kế Cart Service	17
2.6.1	Biểu đồ lớp	17
2.6.2	Data Model	18
2.7	Thiết kế Order Service	18
2.7.1	Biểu đồ lớp	18
2.7.2	Data Model	18
2.7.3	Luồng nghiệp vụ	19
2.8	Thiết kế Payment Service & Shipping Service	19
2.8.1	Biểu đồ lớp	19
2.8.2	Data Model	19
2.9	Sơ đồ ORM của hệ thống	20
2.10	ERD của hệ thống	21
2.11	Biểu đồ luồng xử lý — Use Case Mua hàng (End-to-End)	23
2.12	Checklist đánh giá Chương 2	26
2.13	Kết luận Chương 2	27
3	CHƯƠNG 3: AI SERVICE CHO TƯ VẤN SẢN PHẨM	27
3.1	Mục tiêu và Bối cảnh	27
3.2	Kiến trúc tổng thể AI Service	27
3.3	Dữ liệu đầu vào	28
3.4	Pipeline RAG Chat	28
3.5	Cấu trúc mã nguồn và dữ liệu thử nghiệm	29
3.6	Thành phần 1 — Mô hình Deep Learning cục bộ	30
3.6.1	Lý do lựa chọn	30
3.6.2	Kiến trúc mô hình chi tiết	30
3.6.3	Quy trình huấn luyện và metadata	31
3.7	Thành phần 2 — Knowledge Graph với Neo4j	31
3.7.1	Lý do chọn Graph Database	31
3.7.2	Thiết kế Graph Schema	32

3.8	Thành phần 3 — RAG (Retrieval-Augmented Generation)	33
3.8.1	Tại sao cần RAG?	33
3.8.2	Kiến trúc Multi-Retrieval	33
3.8.3	LLM Router Agent	34
3.9	Kết hợp Hybrid — Tích hợp ba thành phần	35
3.10	Đánh giá & Triển khai	36
3.11	Tích hợp hệ thống Chat với dữ liệu	37
3.12	Tích hợp AI vào hệ thống thương mại điện tử	38
3.13	Triển khai AI Service vào hệ thống Microservices	38
3.14	Checklist đánh giá Chương 3	39
3.15	Kết luận Chương 3	39
4	CHƯƠNG 4: XÂY DỰNG HỆ THỐNG HOÀN CHỈNH	39
4.1	Kiến trúc tổng thể hệ thống	39
4.1.1	Tổng quan	39
4.1.2	Cấu trúc thư mục dự án	40
4.2	API Gateway — Nginx	40
4.2.1	Vai trò và trách nhiệm	40
4.2.2	Cấu hình gateway trong frontend	41
4.3	Authentication — JWT	42
4.3.1	Luồng xác thực hiện tại	42
4.3.2	Cài đặt JWT trong User Service (hướng mở rộng)	42
4.4	Giao tiếp giữa các Service	42
4.4.1	Pattern gọi API nội bộ có xử lý lỗi	42
4.5	Luồng mua hàng End-to-End — Code hoàn chỉnh	43
4.5.1	Luồng nghiệp vụ ở frontend	43
4.5.2	Xử lý trong Order Service và Payment Service	44
4.6	Minh họa giao diện frontend	44
4.7	Docker hoá toàn bộ hệ thống	48
4.7.1	Dockerfile chuẩn cho Django Services	48
4.7.2	docker-compose.yml hoàn chỉnh	48
4.8	Demo kết quả — Minh họa API calls	49
4.8.1	Khởi động hệ thống	49
4.8.2	Demo luồng mua hàng bằng curl	49
4.9	Logging và Monitoring	50
4.9.1	Cấu hình Logging tập trung (Django)	50
4.9.2	Monitoring với Prometheus + Grafana	50
4.10	Đánh giá hệ thống	50
4.10.1	Hiệu năng	50
4.10.2	Khả năng mở rộng	51
4.10.3	Ưu điểm hệ thống	51

4.10.4 Nhược điểm và hướng cải thiện	51
4.11 Checklist đánh giá Chương 4	51
4.12 Kết luận Chương 4 & Toàn bộ tiểu luận	52

CHƯƠNG 1: TỪ MONOLITHIC ĐẾN MICROSERVICES VÀ DDD

1.1. Giới thiệu Monolithic Architecture

1.1.1. Khái niệm

Monolithic Architecture là mô hình kiến trúc phần mềm trong đó toàn bộ các thành phần của hệ thống — bao gồm giao diện người dùng (Presentation Layer), logic nghiệp vụ (Business Logic Layer) và tầng truy xuất dữ liệu (Data Access Layer) — được tích hợp, xây dựng và triển khai như một khối thống nhất duy nhất. Đây là mô hình truyền thống, phổ biến trong giai đoạn đầu của kỹ nghệ phần mềm.

1.1.2. Cấu trúc điển hình

Một hệ thống Monolithic điển hình bao gồm ba tầng chính được đóng gói trong cùng một đơn vị triển khai:

- Presentation Layer: Xử lý giao tiếp với người dùng cuối thông qua giao diện web hoặc ứng dụng di động.
- Business Logic Layer: Chứa toàn bộ quy tắc nghiệp vụ, luồng xử lý và điều phối dữ liệu.
- Data Access Layer: Đảm nhiệm việc truy vấn, lưu trữ và quản lý dữ liệu từ cơ sở dữ liệu trung tâm.

1.1.3. Ví dụ thực tế

Một hệ thống e-commerce xây dựng theo kiến trúc Monolithic sẽ tích hợp toàn bộ các chức năng — quản lý sản phẩm, giỏ hàng, thanh toán, quản lý người dùng và giao hàng — trong cùng một codebase duy nhất. Khi hệ thống cần cập nhật module thanh toán, toàn bộ ứng dụng phải được build lại và triển khai lại, kể cả những module không có thay đổi.

1.1.4. Nhược điểm chi tiết

- Khó mở rộng (Poor Scalability): Khi một module chịu tải cao (ví dụ: module tìm kiếm sản phẩm), hệ thống buộc phải nhân bản (scale) toàn bộ ứng dụng thay vì chỉ scale module đó, gây lãng phí tài nguyên.
- Coupling cao (Tight Coupling): Các module phụ thuộc chặt chẽ lẫn nhau. Một thay đổi nhỏ trong logic nghiệp vụ có thể gây ra hiệu ứng dây chuyền, ảnh hưởng đến toàn bộ hệ thống.
- Rủi ro khi triển khai (Deployment Risk): Mỗi lần phát hành phiên bản mới đều yêu cầu dừng và khởi động lại toàn hệ thống. Một lỗi nhỏ trong một module có thể làm sập toàn bộ dịch vụ.

- Khó phát triển nhóm (Team Scalability): Khi nhiều nhóm phát triển cùng làm việc trên một codebase, xung đột mã nguồn (merge conflict) xảy ra thường xuyên, làm chậm tiến độ.
- Hạn chế công nghệ (Technology Lock-in): Toàn bộ hệ thống bị ràng buộc với một ngôn ngữ lập trình và framework duy nhất.

1.1.5. Khi nào nên dùng Monolithic

Mặc dù có nhiều hạn chế, Monolithic vẫn phù hợp trong các tình huống:

- Hệ thống nhỏ, đơn giản với yêu cầu ít thay đổi.
- Giai đoạn MVP (Minimum Viable Product) cần ra mắt nhanh.
- Team phát triển ít thành viên, chưa có kinh nghiệm quản lý hệ thống phân tán.

1.2. Microservices Architecture

1.2.1. Khái niệm

Microservices Architecture là mô hình kiến trúc phần mềm trong đó hệ thống được phân rã thành tập hợp các dịch vụ nhỏ (services), mỗi dịch vụ thực hiện một chức năng nghiệp vụ riêng biệt, được triển khai độc lập và giao tiếp với nhau thông qua các giao thức chuẩn như REST API hoặc Message Queue.

1.2.2. Đặc điểm cốt lõi

- Mỗi service sở hữu cơ sở dữ liệu riêng biệt (Database-per-Service pattern).
- Giao tiếp giữa các service thông qua API (REST/gRPC) hoặc hệ thống hàng đợi thông điệp (Message Broker).
- Mỗi service có thể được triển khai, nâng cấp và mở rộng độc lập mà không ảnh hưởng đến các service khác.

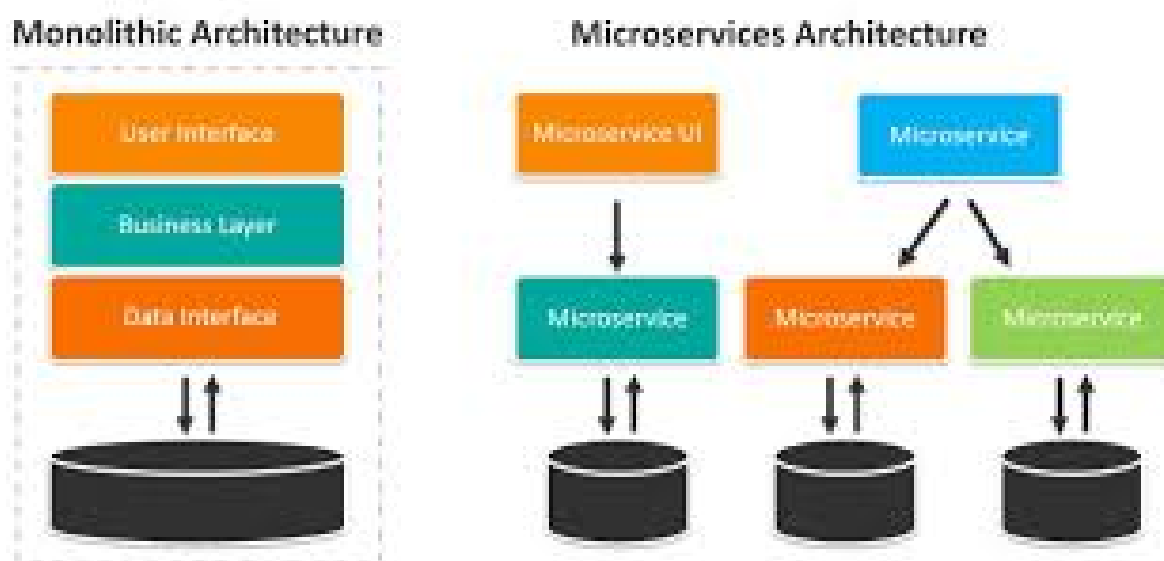
1.2.3. So sánh Monolithic và Microservices

Tiêu chí	Monolithic	Microservices
Đơn vị triển khai	Toàn bộ ứng dụng	Từng service độc lập
Mở rộng (Scale)	Scale toàn hệ thống	Scale từng service riêng lẻ
Mức độ phụ thuộc	Coupling cao	Loose Coupling
Công nghệ	Một ngôn ngữ/framework	Đa ngôn ngữ, đa framework
Độ phức tạp vận hành	Thấp	Cao

Phù hợp với

Hệ thống nhỏ, MVP

Hệ thống lớn, nhiều team



Hình 1: So sánh trực quan giữa Monolithic và Microservices

Hình minh họa trên bổ sung trực quan cho bảng so sánh ở trên. Phần Monolithic cho thấy toàn bộ giao diện, business logic và data layer cùng nằm trong một khối ứng dụng và truy cập chung một cơ sở dữ liệu. Trong khi đó, phần Microservices tách hệ thống thành nhiều service nhỏ, mỗi service có phạm vi trách nhiệm riêng và có thể giao tiếp với nhau qua các lớp trung gian. Sự khác biệt này là nền tảng để giải thích vì sao các chương sau của báo cáo chuyển sang phân rã theo bounded context và service độc lập.

1.2.4. Ưu điểm của Microservices

- Mở rộng độc lập: Chỉ cần tăng tài nguyên cho service đang chịu tải, không cần scale toàn hệ thống.
- Độ tin cậy cao: Lỗi trong một service được cô lập, không lan sang các service khác (Fault Isolation).
- Tăng tốc phát triển: Các nhóm phát triển khác nhau có thể làm việc song song trên từng service mà không xung đột.
- Linh hoạt công nghệ: Mỗi service có thể chọn ngôn ngữ, framework và cơ sở dữ liệu phù hợp nhất với yêu cầu nghiệp vụ của nó.

1.2.5. Nhược điểm của Microservices

- Phức tạp hệ thống: Đòi hỏi cơ sở hạ tầng phức tạp hơn (container orchestration, service discovery, load balancing).

- Khó debug: Việc truy vết lỗi qua nhiều service đòi hỏi hệ thống logging và tracing tập trung.
- Overhead giao tiếp: Mỗi lời gọi API giữa các service đều có độ trễ mạng, cần xử lý các tình huống timeout và retry.

1.2.6. Nguyên tắc thiết kế

- Single Responsibility: Mỗi service chỉ đảm nhiệm một chức năng nghiệp vụ duy nhất.
- Loose Coupling: Các service tương tác với nhau chỉ thông qua interface (API), không truy cập trực tiếp vào nội bộ hoặc cơ sở dữ liệu của service khác.
- High Cohesion: Tất cả logic liên quan đến một nghiệp vụ được gom về cùng một service.

1.3. Domain Driven Design (DDD)

1.3.1. Mục tiêu

Domain Driven Design (DDD) là phương pháp luận thiết kế phần mềm tập trung vào việc mô hình hóa hệ thống dựa trên nghiệp vụ thực tế (business domain), đảm bảo sự đồng nhất giữa ngôn ngữ của chuyên gia nghiệp vụ và mã nguồn của lập trình viên (Ubiquitous Language). DDD không phụ thuộc vào công nghệ cụ thể và đặc biệt hiệu quả khi áp dụng để xác định ranh giới phân rã trong kiến trúc Microservices.

1.3.2. Các khái niệm cốt lõi

- Entity: Đối tượng có định danh (ID) duy nhất và vòng đời riêng biệt. Hai entity khác nhau dù có cùng thuộc tính vẫn là hai đối tượng khác nhau nếu ID khác nhau. Ví dụ: User (user_id), Product (product_id), Order (order_id).
- Value Object: Đối tượng không có định danh riêng, được nhận dạng hoàn toàn bằng giá trị của các thuộc tính. Ví dụ: Address (street, city, country), Money (amount, currency).
- Aggregate: Một nhóm các Entity và Value Object có liên quan chặt chẽ, được xử lý như một đơn vị nhất quán. Mỗi Aggregate có một Aggregate Root là điểm truy cập duy nhất từ bên ngoài. Ví dụ: Order Aggregate gồm Order (root) + danh sách OrderItem.
- Bounded Context: Ranh giới ngữ nghĩa rõ ràng trong đó một mô hình domain cụ thể có giá trị và nhất quán. Cùng một khái niệm (ví dụ: “Product”) có thể có ý nghĩa khác nhau trong các Bounded Context khác nhau. Ví dụ: “Product” trong Product Context chứa thông tin chi tiết kỹ thuật, trong Order Context chỉ cần product_id và giá.

1.3.3. Context Map

Context Map là sơ đồ thể hiện mối quan hệ giữa các Bounded Context trong hệ thống. Hai mẫu quan hệ phổ biến:

- **Shared Kernel:** Hai context chia sẻ một tập con mô hình domain chung, được thống nhất và duy trì bởi cả hai team.
- **Customer-Supplier:** Một context (Supplier) cung cấp dữ liệu/dịch vụ cho context khác (Customer). Ví dụ: Order Context phụ thuộc vào Product Context để lấy thông tin sản phẩm.

1.3.4. DDD trong Microservices

Nguyên tắc cốt lõi: Một Bounded Context tương ứng với một Microservice. Cách ánh xạ này đảm bảo rằng mỗi service được phân rõ dựa trên ranh giới nghiệp vụ tự nhiên, thay vì dựa trên tầng kỹ thuật (technical layer), tránh tạo ra các service quá nhỏ hoặc phụ thuộc chéo phức tạp.

1.4. Case Study: Hệ thống Quản lý Bệnh viện (Healthcare)

1.4.1. Mô tả bài toán

Một hệ thống quản lý bệnh viện cần đáp ứng các nhu cầu nghiệp vụ sau: quản lý thông tin bệnh nhân, quản lý thông tin bác sĩ và điều dưỡng, và điều phối lịch khám bệnh giữa bệnh nhân và bác sĩ. Yêu cầu hệ thống phải có khả năng mở rộng theo từng module khi số lượng bệnh nhân tăng đột biến.

1.4.2. Quy trình phân rã theo DDD

Bước 1 — Xác định Domain chính:

Phân tích nghiệp vụ xác định ba domain cốt lõi:

- **Patient Management Domain:** Quản lý toàn bộ thông tin và hồ sơ y tế của bệnh nhân.
- **Doctor Management Domain:** Quản lý thông tin chuyên môn, lịch làm việc và chuyên khoa của bác sĩ.
- **Appointment Scheduling Domain:** Điều phối, đặt lịch và quản lý trạng thái các ca khám.

Bước 2 — Xác định Bounded Context:

Mỗi domain được đặt trong một Bounded Context riêng biệt với ngôn ngữ nghiệp vụ nhất quán nội bộ:

- **Patient Context:** Khái niệm “bệnh nhân” bao gồm ID, hồ sơ sức khỏe, lịch sử điều trị.
- **Doctor Context:** Khái niệm “bác sĩ” bao gồm ID, chuyên khoa, lịch trực.
- **Appointment Context:** Khái niệm “lịch hẹn” liên kết patient_id và doctor_id, không sở hữu trực tiếp thông tin chi tiết của hai bên.

Bước 3 — Phân rã thành Microservices:

Bounded Context	Microservice	Trách nhiệm
-----------------	--------------	-------------

Patient Context	patient-service	CRUD thông tin bệnh nhân, hồ sơ y tế
Doctor Context	doctor-service	CRUD thông tin bác sĩ, lịch làm việc
Appointment Context	appointment-service	Đặt lịch, xác nhận, hủy, tra cứu lịch hẹn

Bước 4 — Xác định quan hệ giữa các service:

Appointment Service đóng vai trò Customer, phụ thuộc vào Patient Service và Doctor Service (cả hai đóng vai trò Supplier). Giao tiếp được thực hiện qua REST API, không truy cập trực tiếp vào cơ sở dữ liệu của nhau.

1.4.3. Ví dụ API

- Patient Service
- GET /patients/{id} → Lấy thông tin bệnh nhân
- POST /patients → Đăng ký bệnh nhân mới
- Doctor Service
- GET /doctors/{id} → Lấy thông tin bác sĩ
- GET /doctors/{id}/schedule → Xem lịch làm việc
- Appointment Service
- POST /appointments → Đặt lịch khám mới
- GET /appointments/{id} → Tra cứu lịch hẹn
- PUT /appointments/{id}/cancel → Hủy lịch hẹn

1.5. Kết luận Chương 1

Monolithic Architecture phù hợp với các hệ thống nhỏ, đội phát triển ít thành viên và giai đoạn MVP cần ra mắt nhanh. Khi hệ thống tăng trưởng về quy mô và độ phức tạp, Microservices Architecture trở thành lựa chọn tất yếu nhờ khả năng mở rộng độc lập và tính linh hoạt cao. Domain Driven Design cung cấp phương pháp luận khoa học để xác định đúng ranh giới phân rã, đảm bảo mỗi Microservice phản ánh chính xác một miền nghiệp vụ thực tế — đây là nền tảng thiết kế không thể thiếu trong các hệ thống phần mềm quy mô lớn hiện đại.

CHƯƠNG 2: PHÁT TRIỂN HỆ THỐNG E-COMMERCE MICROSERVICE

2.1. Xác định yêu cầu hệ thống

2.1.1. Yêu cầu chức năng (*Functional Requirements*)

Hệ thống e-commerce được tổ chức xoay quanh các nhóm chức năng cốt lõi sau. Những phần như Cart, Shipping, JWT và Nginx được trình bày ở mức logic hoặc hướng mở rộng nếu chưa tách service riêng.

Nhóm chức năng	Mô tả
Quản lý sản phẩm	Hỗ trợ catalog sản phẩm với SKU, danh mục, thương hiệu, giá, giảm giá, tồn kho, rating, thuộc tính mở rộng và trạng thái hiển thị.
Quản lý người dùng	Quản lý hồ sơ người dùng theo vai trò admin , customer , staff ; đăng nhập demo bằng email và xem summary theo role.
Giỏ hàng	Chức năng giữ item trước checkout được xử lý ở tầng frontend và payload tạo order; trong repo hiện tại chưa tách Cart Service độc lập.
Đặt hàng	Tạo đơn từ danh sách item, kiểm tra user/product qua service liên quan và lưu snapshot sản phẩm tại thời điểm mua.
Thanh toán	Tạo payment dựa trên order đã tồn tại, lưu phương thức, trạng thái, mã giao dịch và phản hồi gateway demo.
Giao hàng	Phí ship và địa chỉ giao hàng được lưu trong Order ; Shipping Service riêng là hướng mở rộng.
Gợi ý AI	Tích hợp ai_service để chat, search, dự báo xu hướng và truy vấn dữ liệu nghiệp vụ.

2.1.2. Yêu cầu phi chức năng (*Non-functional Requirements*)

- **Scalability:** Mỗi service có thể được mở rộng độc lập tùy theo tải thực tế.
- **High Availability:** Lỗi ở một service không được làm gián đoạn toàn bộ luồng nghiệp vụ.
- **Security:** Hệ thống hiện mới ở mức login demo theo email; JWT và RBAC chi tiết là hướng mở rộng phù hợp với kiến trúc microservices.
- **Maintainability:** Codebase tách biệt theo service, dễ bảo trì và nâng cấp độc lập.

- **Observability:** Hệ thống có summary endpoint và healthcheck; logging/monitoring chi tiết là nền tảng để phát triển tiếp.

2.2. Phân rã hệ thống theo DDD

2.2.1. Bounded Context và ánh xạ Service

Bounded Context	Service / Module	Trách nhiệm
User Context	<code>user_service</code>	Quản lý hồ sơ người dùng, vai trò, trạng thái, đăng nhập demo và summary.
Product Context	<code>product_service</code>	Quản lý catalog sản phẩm, danh mục, tồn kho, giá, rating và summary catalog.
Cart Context	<code>client</code> / logic tạm	Giữ item trước checkout trong frontend hoặc payload tạo order; chưa tách service riêng.
Order Context	<code>order_service</code>	Tạo đơn hàng, validate user/product, lưu order item và snapshot sản phẩm.
Payment Context	<code>payment_service</code>	Tạo giao dịch thanh toán từ order và lưu metadata demo.
Shipping Context	<code>order_service</code> / mở rộng	Thông tin giao hàng, phí ship và trạng thái giao hàng hiện được gộp vào Order; tách service riêng là hướng phát triển tiếp theo.
AI Context	<code>ai_service</code>	Intent classification, retrieval, Neo4j và chatbot.
API Composition	<code>gateway</code>	Điểm vào backend, proxy request tới các service.
Presentation	<code>client</code>	Dashboard, quản lý, tạo đơn, login và chatbot.

2.2.2. Nguyên tắc thiết kế

Repo hiện tại áp dụng tinh thần DDD ở mức phù hợp cho demo:

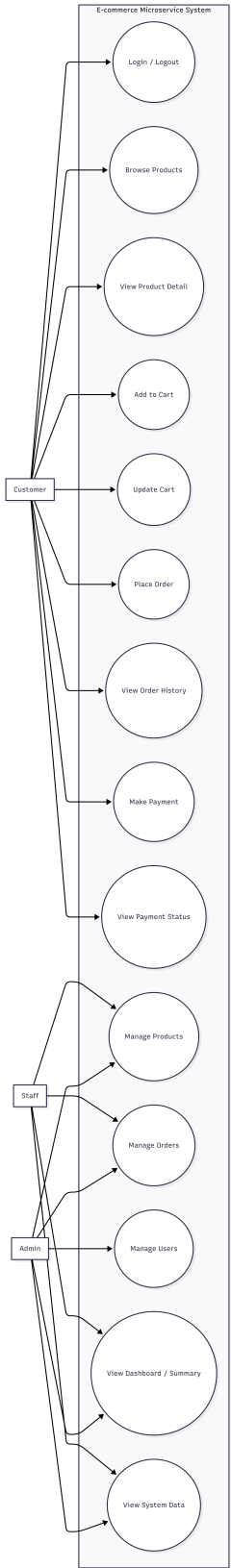
- Mỗi bounded context có ranh giới rõ ràng và được ánh xạ sang một service riêng hoặc một module logic rõ ràng.

- Không dùng foreign key xuyên service; các service xác thực dữ liệu thông qua REST API.
- Dữ liệu quan trọng như sản phẩm trong đơn hàng phải được snapshot để giữ nguyên lịch sử giao dịch.
- Gateway là điểm vào thống nhất và đóng vai trò proxy, giúp client không phụ thuộc vào vị trí thực của từng service.

2.3. Biểu đồ Use Case nghiệp vụ

Biểu đồ dưới đây mô tả các use case nghiệp vụ chính của hệ thống e-commerce theo góc nhìnDDD. Ba actor trung tâm là **Customer**, **Staff** và **Admin**. Nhóm use case của **Customer** bao phủ toàn bộ luồng mua hàng cơ bản như đăng nhập, xem sản phẩm, thêm giỏ hàng, đặt hàng, xem lịch sử đơn và thanh toán. Nhóm use case của **Staff** và **Admin** tập trung vào các nghiệp vụ vận hành như quản lý sản phẩm, đơn hàng, người dùng và dashboard tổng quan.

Sơ đồ này hữu ích cho phầnDDD vì nó giúp xác định ranh giới nghiệp vụ trước khi ánh xạ sang bounded context và service. Từ use case, có thể thấy rõ hệ thống đang xoay quanh bốn miền chính: User, Product, Order và Payment. AI service không xuất hiện trong sơ đồ này để giữ phạm vi đúng với phầnDDD của chương 2, nơi trọng tâm là phân rã nghiệp vụ cốt lõi của hệ thống giao dịch.



Hình 2: Biểu đồ use case nghiệp vụ của hệ thống e-commerce

2.4. Thiết kế Product Service

2.4.1. Biểu đồ lớp (Class Diagram)

Product là thực thể trung tâm của catalog. Mô hình hiện tại dùng category như một thuộc tính của Product, còn các chi tiết mở rộng được lưu trong JSONField.

- Product là aggregate root.
- sku và slug dùng để định danh nghiệp vụ.
- status mô tả trạng thái hiển thị của sản phẩm.
- attributes và dimensions cho phép mở rộng theo từng nhóm hàng mà không phải sửa schema nhiều lần.

2.4.2. Data Model (sinh từ Class Diagram)

Listing 1: Data model rút gọn của Product Service

```
CREATE TABLE product (
  id          SERIAL PRIMARY KEY,
  sku         VARCHAR(64) UNIQUE NOT NULL,
  name        VARCHAR(255) NOT NULL,
  slug        VARCHAR(255) UNIQUE NOT NULL,
  category     VARCHAR(120) NOT NULL,
  brand        VARCHAR(120) NOT NULL,
  price        DECIMAL(12, 2) NOT NULL,
  discount_price DECIMAL(12, 2),
  currency     VARCHAR(10) NOT NULL DEFAULT 'VND',
  stock_quantity INTEGER NOT NULL DEFAULT 0,
  safety_stock INTEGER NOT NULL DEFAULT 0,
  rating        DECIMAL(3, 2) NOT NULL DEFAULT 0,
  weight_grams  INTEGER NOT NULL DEFAULT 0,
  dimensions   JSONB NOT NULL DEFAULT '{}',
  color         VARCHAR(64),
  material      VARCHAR(120),
  size          VARCHAR(32),
  origin_country VARCHAR(120),
  warranty_months INTEGER NOT NULL DEFAULT 0,
  description   TEXT,
  tags          JSONB NOT NULL DEFAULT '[]',
  attributes    JSONB NOT NULL DEFAULT '{}',
  thumbnail_url TEXT,
  status        VARCHAR(20) NOT NULL DEFAULT 'active',
  published_at  TIMESTAMP NULL,
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
```



```
updated_at      TIMESTAMP NOT NULL DEFAULT NOW()
);
```

2.4.3. API

Method	Endpoint	Mô tả
GET	/api/products/	Lấy danh sách sản phẩm; hỗ trợ filter theo category.
POST	/api/products/	Tạo sản phẩm mới.
GET	/api/products/<id>/	Lấy chi tiết một sản phẩm.
GET	/api/products/summary/	Tổng hợp số lượng sản phẩm, rating trung bình và danh sách danh mục.

2.5. Thiết kế User Service

2.5.1. Biểu đồ lớp (Class Diagram)

Trong repo hiện tại, User Service dùng `UserProfile` thay vì kế thừa `AbstractUser`. Mô hình này phù hợp với đồ án vì tập trung vào hồ sơ người dùng và các thuộc tính nghiệp vụ hơn là cơ chế auth đầy đủ.

- `UserProfile` là entity trung tâm.
- `Role` và `Status` là các `TextChoices` tương đương enum nghiệp vụ.
- `metadata` cho phép lưu thông tin mở rộng mà không làm phình schema.

2.5.2. Data Model

Listing 2: Data model rút gọn của User Service

```
CREATE TABLE user_profile (
  id          SERIAL PRIMARY KEY,
  full_name   VARCHAR(255) NOT NULL,
  email       VARCHAR(255) UNIQUE NOT NULL,
  phone       VARCHAR(20),
  role        VARCHAR(20) NOT NULL DEFAULT 'customer',
  status      VARCHAR(20) NOT NULL DEFAULT 'active',
  loyalty_points INTEGER NOT NULL DEFAULT 0,
  address     VARCHAR(255),
  city        VARCHAR(120),
  country     VARCHAR(120) NOT NULL DEFAULT 'Vietnam',
  avatar_url  TEXT,
```

```

metadata      JSONB NOT NULL DEFAULT '{}',
created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
updated_at    TIMESTAMP NOT NULL DEFAULT NOW()
);

```

2.5.3. Phân quyền RBAC

Vai trò	Quyền hạn
Admin	Quản lý toàn bộ tài nguyên hệ thống, quản lý catalog, theo dõi dashboard và dữ liệu vận hành.
Staff	Xem và xử lý dữ liệu vận hành, hỗ trợ kiểm tra sản phẩm, đơn hàng và thanh toán.
Customer	Xem sản phẩm, đăng nhập demo, tạo đơn hàng và tương tác với chatbot AI.

2.5.4. API

Method	Endpoint	Mô tả
GET	/api/users/	Lấy danh sách người dùng; hỗ trợ lọc theo role.
POST	/api/users/	Tạo người dùng mới.
POST	/api/users/login/	Đăng nhập demo bằng email, kiểm tra trạng thái active.
GET	/api/users/summary/	Đếm tổng số user và phân bố theo role.
GET	/api/users/<id>/	Lấy chi tiết một người dùng.

2.6. Thiết kế Cart Service

2.6.1. Biểu đồ lớp

Cart Service là một bounded context logic với **Cart** và **CartItem**. Trong hệ thống hiện tại, chức năng này chưa được tách thành microservice độc lập. Thay vào đó:

- Frontend giữ trạng thái chọn sản phẩm trước khi checkout.
- Payload tạo order đóng vai trò “giỏ hàng tạm” khi gửi xuống backend.

Nếu tách riêng theo mô hình đầy đủ hơn, biểu đồ lớp logic sẽ có dạng:

Listing 3: Mô hình logic Cart Service

```

Cart 1 --- * CartItem

```

2.6.2. Data Model

Hệ thống hiện chưa có bảng `cart` và `cart_item`. Nếu mở rộng theo hướng tách Cart Service, hai bảng này sẽ lưu trạng thái giỏ hàng tạm và số lượng item trước khi checkout.

2.7. Thiết kế Order Service

2.7.1. Biểu đồ lớp

Order Service là nơi ràng buộc nghiệp vụ quan trọng nhất của hệ thống. Order được tạo trực tiếp từ payload item do frontend chuẩn bị sẵn. Hai entity chính là `Order` và `OrderItem`.

- `Order` là aggregate root.
- `OrderItem` mang snapshot của sản phẩm tại thời điểm mua.
- Trạng thái order trong repo: `pending`, `confirmed`, `shipping`, `completed`, `cancelled`.

2.7.2. Data Model

Listing 4: Data model rút gọn của Order Service

```
CREATE TABLE orders (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER NOT NULL,
  customer_name VARCHAR(255) NOT NULL,
  customer_email VARCHAR(255) NOT NULL,
  shipping_address VARCHAR(255) NOT NULL,
  shipping_city VARCHAR(120) NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'pending',
  subtotal_amount DECIMAL(12, 2) NOT NULL,
  shipping_fee DECIMAL(12, 2) NOT NULL DEFAULT 0,
  total_amount DECIMAL(12, 2) NOT NULL,
  notes       TEXT,
  metadata    JSONB NOT NULL DEFAULT '{}',
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE order_item (
  id          SERIAL PRIMARY KEY,
  order_id     INTEGER REFERENCES orders(id) ON DELETE CASCADE,
  product_id   INTEGER NOT NULL,
  product_name VARCHAR(255) NOT NULL,
  sku          VARCHAR(64) NOT NULL,
  quantity     INTEGER NOT NULL,
  unit_price   DECIMAL(12, 2) NOT NULL,
```

```

    line_total      DECIMAL(12, 2) NOT NULL,
    product_snapshot JSONB NOT NULL DEFAULT '{}
);

```

2.7.3. Luồng nghiệp vụ

Khi tạo đơn hàng, service sẽ:

1. Kiểm tra người dùng qua `user_service`.
2. Duyệt từng item và kiểm tra sản phẩm qua `product_service`.
3. Tính `subtotal_amount`, lấy `shipping_fee` mặc định hoặc từ payload, rồi cộng ra `total_amount`.
4. Tạo `Order` trước, sau đó ghi các `OrderItem` bằng `bulk_create`.

2.8. Thiết kế Payment Service & Shipping Service

2.8.1. Biểu đồ lớp

Payment Service đã được triển khai. Shipping Service chưa tách riêng thành một microservice, nên phần này được trình bày theo hai lớp:

- **Payment** là phần đã hiện thực.
- **Shipping** là phần gộp vào `Order` và là hướng mở rộng nếu muốn tách riêng theo mô hình nghiệp vụ đầy đủ hơn.

Listing 5: Mô hình logic Payment và Shipping

Payment	Shipping (m rng)
-----	-----
order_id	order_id
amount	address
status	status
paid_at	created_at

2.8.2. Data Model

Listing 6: Data model rút gọn của Payment Service

```

CREATE TABLE payment (
    id          SERIAL PRIMARY KEY,
    order_id    INTEGER NOT NULL UNIQUE,
    payer_name  VARCHAR(255) NOT NULL,
    amount      DECIMAL(12, 2) NOT NULL,
    currency    VARCHAR(10) NOT NULL DEFAULT 'VND',

```

```

method          VARCHAR(20) NOT NULL,
status          VARCHAR(20) NOT NULL DEFAULT 'pending',
transaction_code VARCHAR(64) UNIQUE NOT NULL,
gateway_response JSONB NOT NULL DEFAULT '{}',
paid_at         TIMESTAMP NULL,
created_at      TIMESTAMP NOT NULL DEFAULT NOW(),
updated_at      TIMESTAMP NOT NULL DEFAULT NOW()
);

```

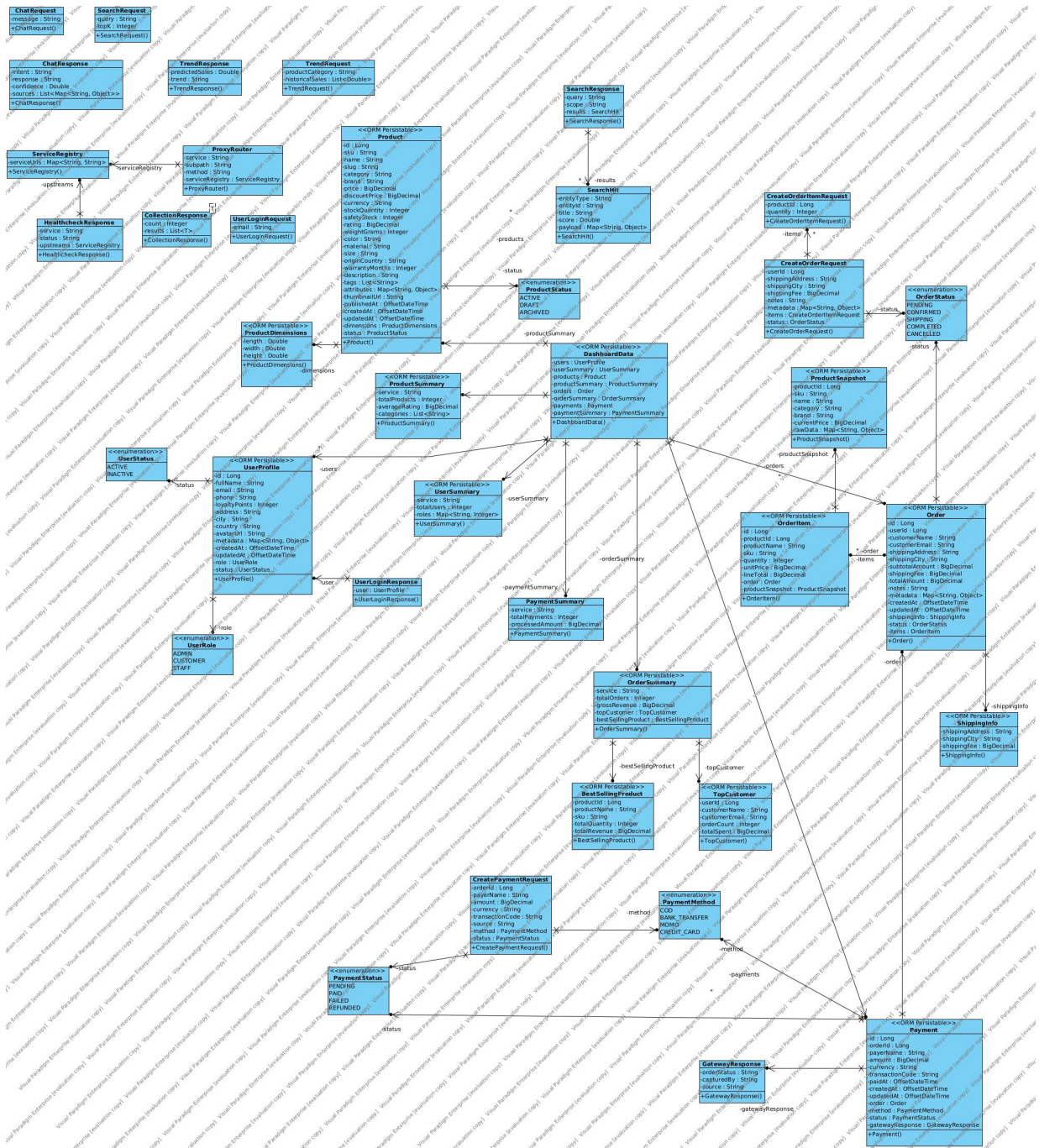
Shipping logic trong code hiện tại được gộp vào `Order` thông qua `shipping_address`, `shipping_city` và `shipping_fee`. Nếu tách riêng theo mô hình tham khảo, có thể bổ sung bảng `shipment` độc lập ở giai đoạn sau.

2.9. Sơ đồ ORM của hệ thống

Sơ đồ ORM cho thấy cách hệ thống hiện thực các khái niệm nghiệp vụ thành model trong mã nguồn. Đây không chỉ là bản đồ của các bảng dữ liệu, mà còn là nơi thể hiện cách từng service tổ chức domain của mình ở mức object. Trong sơ đồ này, `Product` đóng vai trò trung tâm của catalog, `UserProfile` đại diện cho hồ sơ người dùng, `Order` và `OrderItem` tạo thành lõi của luồng mua hàng, còn `Payment` phản ánh bước thanh toán sau khi đơn hàng đã được tạo.

Một điểm đáng chú ý là hệ thống không chỉ có các model nghiệp vụ mà còn có các model phục vụ tổng hợp dữ liệu cho dashboard. Các đối tượng như `DashboardData`, `UserSummary`, `ProductSummary`, `OrderSummary` và `PaymentSummary` đóng vai trò gom kết quả thống kê từ nhiều service thành một cấu trúc dễ trả về cho frontend. Điều này cho thấy kiến trúc không dừng ở CRUD đơn lẻ, mà còn chuẩn bị sẵn lớp dữ liệu phục vụ báo cáo, giám sát và hiển thị tổng quan.

Từ góc nhìn thiết kế, sơ đồ ORM cũng cho thấy cách các model được chia thành hai nhóm rõ ràng. Nhóm thứ nhất là các model cốt lõi gắn trực tiếp với nghiệp vụ giao dịch như sản phẩm, người dùng, đơn hàng và thanh toán. Nhóm thứ hai là các model trung gian hoặc tổng hợp, dùng để ghép dữ liệu từ nhiều nguồn khác nhau. Cách tổ chức này giúp code dễ đọc hơn, dễ mở rộng hơn và phản ánh đúng ranh giới trách nhiệm giữa các phần trong hệ thống.



Hình 3: Sơ đồ ORM tổng hợp của hệ thống

2.10. ERD của hệ thống

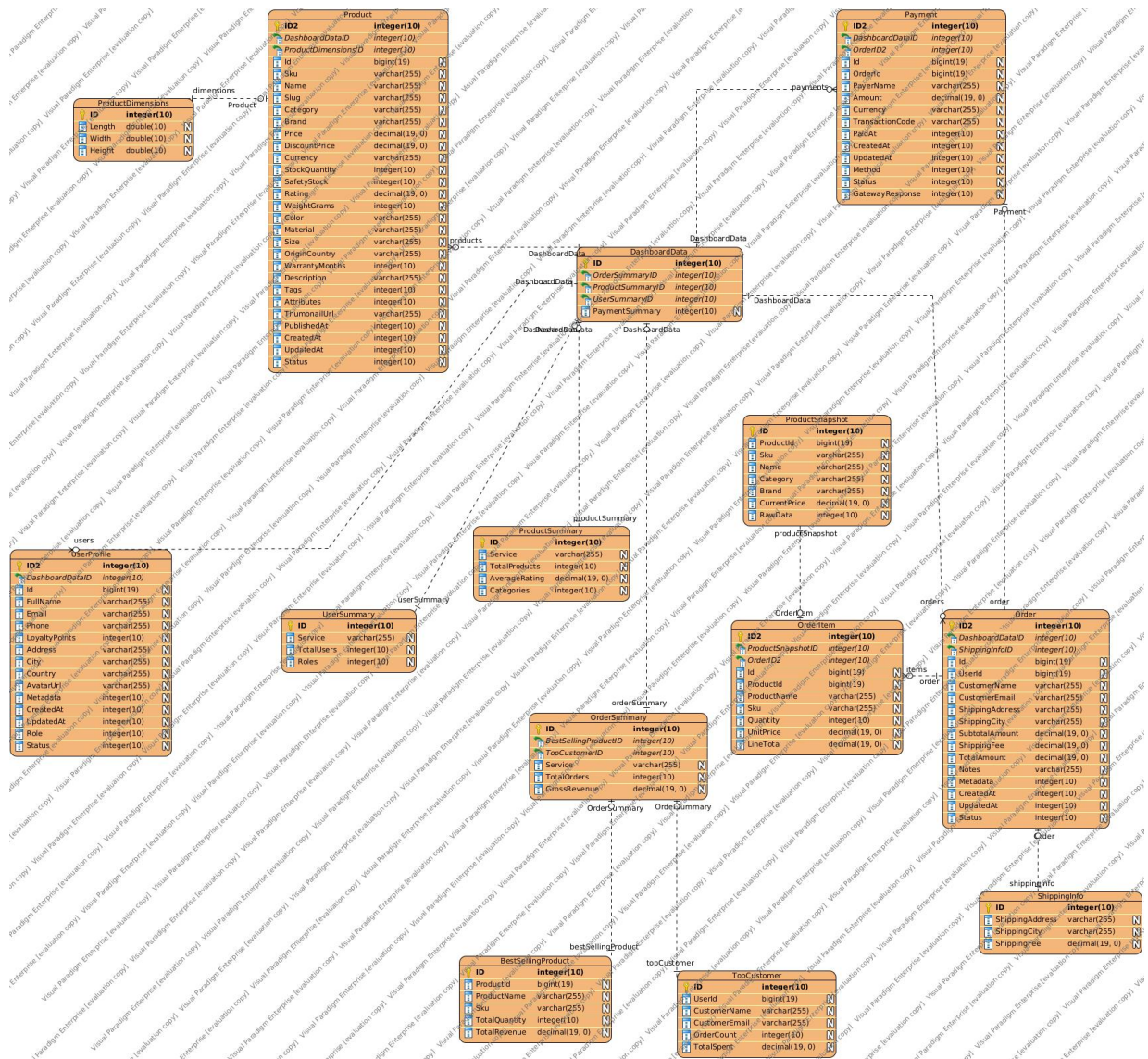
ERD mô tả cấu trúc dữ liệu ở mức quan hệ, tức là lớp mà hệ thống thực sự lưu xuống cơ sở dữ liệu. Nếu ORM cho thấy cách các model được biểu diễn trong code, thì ERD cho thấy dữ liệu được chia thành bảng nào, mỗi bảng giữ vai trò gì và chúng liên kết với nhau ra sao. Đây là phần rất quan trọng vì nó quyết định cách dữ liệu được truy vấn, cập nhật và bảo toàn tính nhất quán.

Trong sơ đồ này, bảng **Product** lưu thông tin catalog đầy đủ như SKU, tên, danh mục, giá, tồn kho, rating, thuộc tính mở rộng và trạng thái. Bảng **UserProfile** giữ hồ sơ người

dùng với role, trạng thái, địa chỉ và dữ liệu mở rộng. Hai bảng **Order** và **OrderItem** thể hiện quan hệ một-nhiều giữa đơn hàng và các sản phẩm nằm trong đơn; trong đó **OrderItem** không chỉ giữ ID sản phẩm mà còn lưu snapshot như tên sản phẩm, SKU, đơn giá và thành tiền tại thời điểm mua để bảo toàn lịch sử giao dịch.

Phần thanh toán được thể hiện qua bảng **Payment**, có liên kết trực tiếp với **Order** thông qua **order_id**. Ngoài ra, sơ đồ còn cho thấy các bảng phụ trợ như **ShippingInfo** và các bảng tổng hợp phục vụ dashboard. **ShippingInfo** lưu địa chỉ và phí giao hàng, trong khi các bảng summary như **DashboardData**, **ProductSummary**, **OrderSummary**, **UserSummary**, **PaymentSummary** đóng vai trò dữ liệu dẫn xuất cho màn hình tổng quan. Điều này chứng tỏ cơ sở dữ liệu của hệ thống không chỉ lưu giao dịch, mà còn hỗ trợ tốt cho việc thống kê và quan sát vận hành.

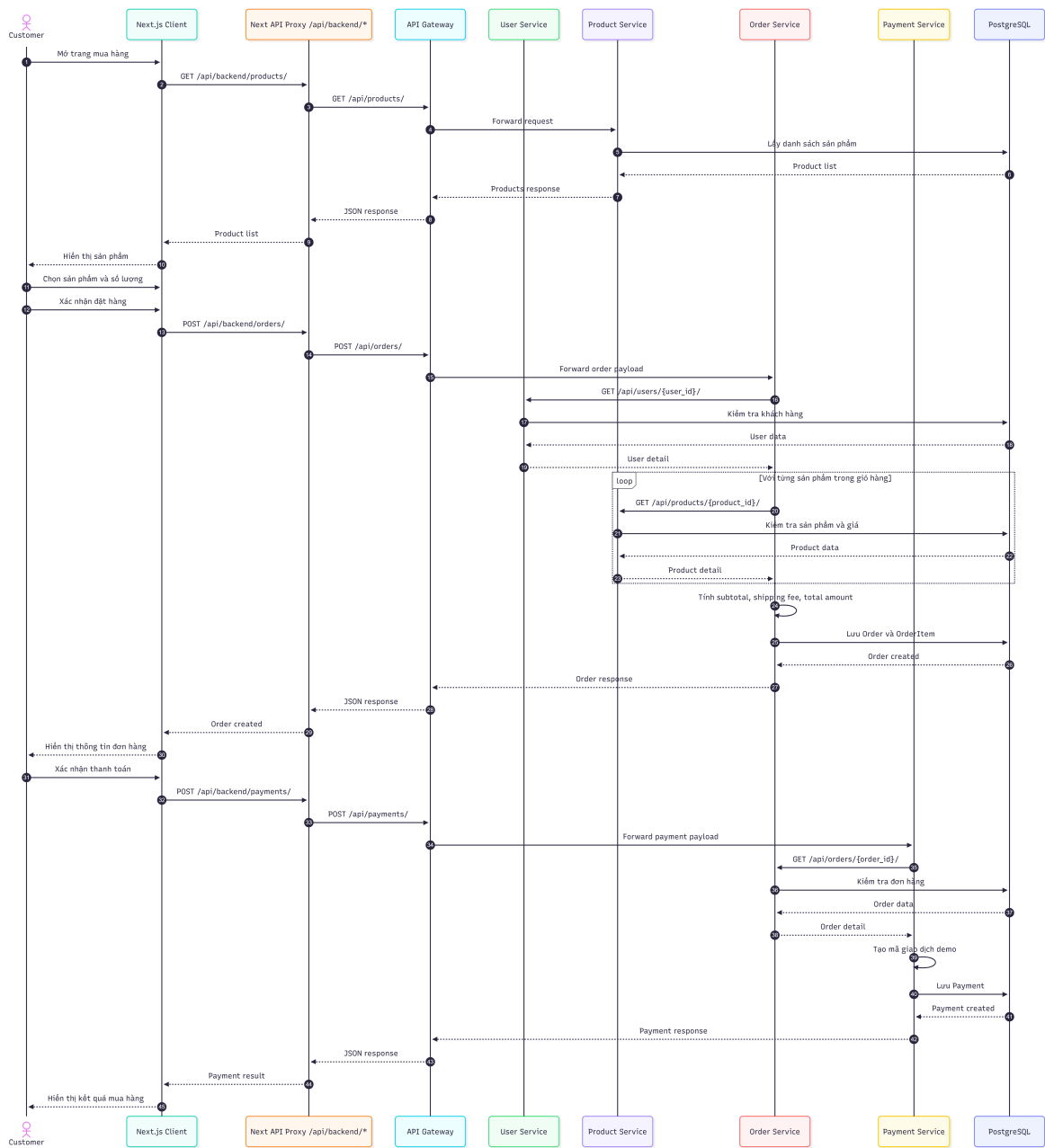
Về mặt thiết kế, ERD giúp làm rõ một nguyên tắc quan trọng của hệ thống: dữ liệu giao dịch phải được lưu đủ chi tiết để có thể truy vết lại sau này. Vì vậy, đơn hàng lưu cả **customer_name**, **customer_email**, **shipping_address**, **subtotal_amount**, **shipping_fee** và **total_amount**; payment lưu **gateway_response**, **transaction_code** và thời điểm thanh toán. Những trường này biến ERD thành nền tảng không chỉ cho thao tác nghiệp vụ trước mắt mà còn cho kiểm tra, đối soát và mở rộng sau này.



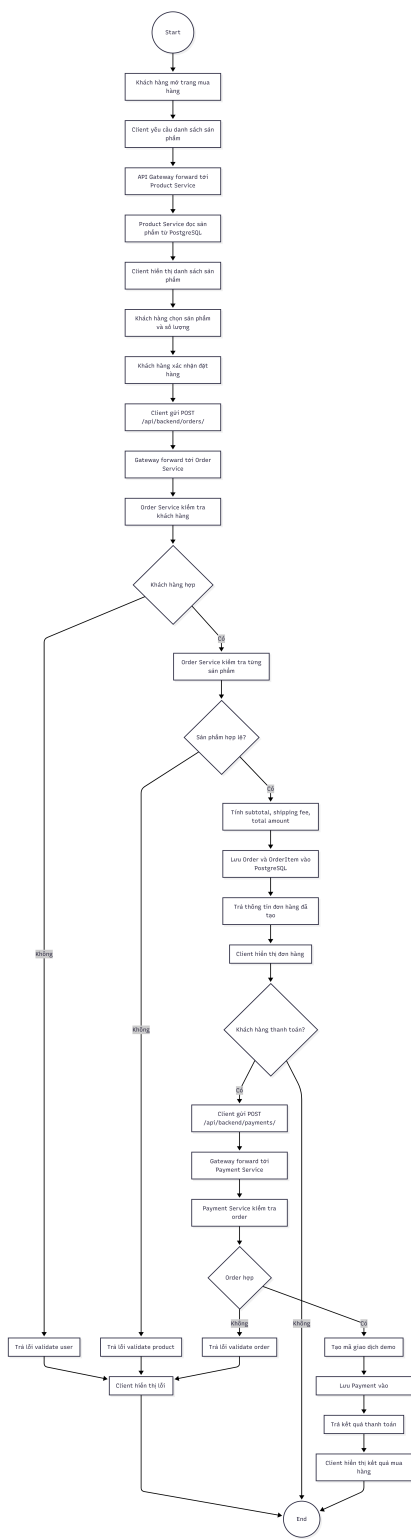
Hình 4: ERD tổng quan của hệ thống

2.11. Biểu đồ luồng xử lý — Use Case Mua hàng (End-to-End)

Phần này mô tả luồng mua hàng end-to-end của hệ thống, từ lúc khách hàng xem sản phẩm đến khi tạo đơn hàng và thanh toán:



Hình 5: Sequence diagram luồng mua hàng end-to-end



Hình 6: Activity diagram luồng mua hàng end-to-end

Bước	Đối tượng	Hành động
1	Client → Gateway	Gọi /api/users/login/ để đăng nhập demo bằng email.

2	Gateway user_service	→	Kiểm tra người dùng, trạng thái active và trả dữ liệu hồ sơ.
3	Client → Gateway		Gọi /api/products/ hoặc /api/products/summary/ để lấy catalog.
4	Gateway product_service	→	Trả danh sách sản phẩm và các trường chi tiết.
5	Client → Gateway		Gửi POST /api/orders/ với user_id và danh sách item.
6	Gateway order_service	→	Tạo order, validate user và từng sản phẩm qua service liên quan.
7	order_service product_service	→	Lấy giá hiện tại và snapshot sản phẩm.
8	order_service Gateway → Client	→	Trả về order đã tạo với status: pending.
9	Client → Gateway		Gửi POST /api/payments/ cho order vừa tạo.
10	Gateway payment_service	→	Validate order, tạo payment và cập nhật trạng thái giao dịch.
11	payment_service order_service	→	Tra cứu order để lấy thông tin khách hàng và tổng tiền.
12	payment_service Gateway → Client	→	Trả về kết quả thanh toán thành công và trạng thái xử lý tiếp theo.

Cart Service và Shipping Service chưa tách riêng, nên các bước đó được gộp vào luồng order/payment và các trường shipping_fee, shipping_address trong Order.

2.12. Checklist đánh giá Chương 2

Hạng mục	Trạng thái
Yêu cầu chức năng và phi chức năng	Đã trình bày
Phân rã hệ thống theo DDD	Đã trình bày
Thiết kế Product Service	Đã trình bày
Thiết kế User Service	Đã trình bày
Thiết kế Cart Service	Mô hình logic / chưa tách service
Thiết kế Order Service	Đã trình bày
Thiết kế Payment & Shipping	Payment đã hiện thực, Shipping là hướng mở rộng

Luồng mua hàng end-to-end

Đã trình bày

Data Model từng service

Đã trình bày theo code hiện tại

2.13. Kết luận Chương 2

Hệ thống e-commerce vẫn đi theo kiến trúc Microservices với các bounded context chính là User, Product, Order, Payment, AI và lớp gateway/client. Cart và Shipping hiện là mô hình logic hoặc hướng mở rộng, còn Order/Payment/User/Product là phần đã hiện thực đầy đủ. Nhờ vậy, nội dung vừa bám sát kiến trúc hệ thống, vừa phản ánh trung thực những gì code đang có.

CHƯƠNG 3: AI SERVICE CHO TỰ VẤN SẢN PHẨM

3.1. Mục tiêu và Bối cảnh

AI Service được thiết kế để hỗ trợ truy vấn nghiệp vụ bằng ngôn ngữ tự nhiên cho hệ thống e-commerce. Trong repository hiện tại, service này phục vụ chat, tra cứu dữ liệu, healthcheck và dự báo xu hướng đơn giản. Điểm quan trọng là AI không đứng tách rời, mà được đặt đúng vào luồng nghiệp vụ của `user_service`, `product_service` và `order_service`.

3.2. Kiến trúc tổng thể AI Service

AI Service được hiện thực bằng một ứng dụng FastAPI độc lập. Service này có bốn lớp chính:

- Lớp tiếp nhận yêu cầu: `/health`, `/chat`, `/search`, `/trends`.
- Lớp nhận diện intent cục bộ: mô hình `numpy_mlp_text_classifier` và rule-based fallback.
- Lớp truy xuất tri thức: lấy dữ liệu từ các service nghiệp vụ và ưu tiên Neo4j.
- Lớp sinh phản hồi: OpenAI, Gemini hoặc fallback local nếu chưa cấu hình provider.

Thành phần	File chính	Vai trò
Intent model	<code>train_models_kaggle.py</code>	Huấn luyện mô hình intent cục bộ và sinh metadata.
Retriever	<code>retrieval.py</code>	Xây dựng tài liệu RAG, xếp hạng và truy vấn Neo4j.
Neo4j ingest	<code>ingest_to_neo4j.py</code>	Đồng bộ dữ liệu sang đồ thị và tạo vector index.

API runtime	<code>main.py</code>	Cung cấp các endpoint chat, search, trends và healthcheck.
-------------	----------------------	--

3.3. Dữ liệu đầu vào

AI Service lấy dữ liệu từ ba nguồn chính:

- Dataset huấn luyện intent: `ai_service/data/intent_seed_dataset.csv`.
- Trạng thái mô hình: `ai_service/models/intent_mlp_model.npz` và `training_metadata.json`.
- Dữ liệu nghiệp vụ thực tế từ user, product và order service, sau đó đồng bộ sang Neo4j.

3.4. Pipeline RAG Chat

Pipeline chat của hệ thống được thiết kế theo đúng tinh thần `requirement.md`: đầu vào là câu hỏi người dùng, đầu ra là câu trả lời có ngữ cảnh, không bịa dữ liệu và có thể truy vết nguồn.

Bước	Khối xử lý	Mô tả
1	Frontend chat widget	Người dùng nhập câu hỏi từ khung chat nổi ở góc màn hình.
2	Next.js proxy	Request được chuyển qua <code>/api/backend/ai/chat</code> để tránh CORS và giữ backend URL ẩn với UI.
3	AI runtime	<code>handle_chat()</code> resolve intent bằng model local hoặc rule-based fallback.
4	Retrieval layer	Nếu câu hỏi thuộc product/order/user, service truy xuất Neo4j trước, sau đó mới fallback TF-IDF.
5	LLM router	OpenAI, Gemini hoặc local fallback sinh phản hồi JSON chuẩn hoá.
6	Response assembly	AI service gắn thêm <code>sources</code> và trả lại cho frontend.
7	UI rendering	Chatbot hiển thị câu trả lời ngay trong widget, giữ luồng hội thoại liên tục.

Listing 7: Luồng gửi chat từ frontend tới AI service

```
const res = await fetch("/api/backend/ai/chat", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ message: userMessage.text }),
});
```

```
});

const data = await res.json();
setMessages((prev) => [
  ...prev,
  { id: (Date.now() + 1).toString(), sender: "bot", text: data.response || "
    Sorry, I didn't get that." },
]);
```

3.5. Cấu trúc mã nguồn và dữ liệu thử nghiệm

Phần AI service có cấu trúc mã nguồn rõ ràng và kèm dữ liệu thử nghiệm cụ thể. Cấu trúc cốt lõi được chia như sau:

Thư mục / file	Vai trò
ai_service/main.py	Điều phối runtime, intent resolution, RAG chat, search, trends và healthcheck.
ai_service/retrieval.py	Chuyển hoá tài liệu truy xuất, tạo scope và xếp hạng kết quả tìm kiếm.
ai_service/ingest_to_neo4j.py	Điền và cập nhật dữ liệu từ các service sang Neo4j và tạo vector index.
ai_service/train_model_huaggle.py	Huấn luyện mô hình intent local và sinh metadata.
e-commerce-client/src/components/chat/chatbot.tsx	Kiểm tra và chuyển đổi câu hỏi tới AI service.
e-commerce-client/src/app/layout.tsx	Cấp layout toàn cục để dùng ở mọi trang.

Dữ liệu thử nghiệm có thể lấy trực tiếp từ dataset intent hoặc từ ngữ cảnh nghiệp vụ e-commerce. Ba mẫu tiêu biểu:

Câu hỏi	Intent dự kiến	Scope	Ghi chú
where is my order	order_status	orders	Chạy qua retrieval đơn hàng.
show customer profile	customer_lookup	users	Truy xuất hồ sơ khách hàng.

co san pham nao dang ban chay	product_inquiry products	Trả về tài liệu sản phẩm.
----------------------------------	--------------------------	---------------------------------

3.6. Thành phần 1 — Mô hình Deep Learning cục bộ

3.6.1. Lý do lựa chọn

Bài toán intent classification được giải bằng một mô hình NumPy MLP gọn nhẹ. Cách làm này phù hợp với hệ thống vì:

- chạy offline, không phụ thuộc framework nặng;
- huấn luyện nhanh trên dataset nhỏ;
- đủ tốt cho lớp nhận diện intent trước khi chuyển sang RAG hoặc LLM.

3.6.2. Kiến trúc mô hình chi tiết

Mô hình cục bộ dùng vector đặc trưng ký tự/từ, sau đó đi qua hai lớp hidden và một lớp softmax đầu ra. Dữ liệu huấn luyện được chuẩn hoá bằng các nhãn nghiệp vụ như `greeting`, `product_inquiry`, `order_status`, `complaint`, `farewell`, `customer_lookup`, `unknown`.

Listing 8: Rút gọn phần huấn luyện và lưu mô hình local

```
def train_mlp(
    train_df: pd.DataFrame,
    valid_df: pd.DataFrame,
    max_features: int,
    hidden_size: int,
    hidden_size_2: int,
    epochs: int,
    learning_rate: float,
    weight_decay: float,
    random_state: int,
) -> dict:
    rng = np.random.default_rng(random_state)
    labels = sorted(train_df["intent"].unique().tolist())
    vocab = build_vocab(train_df["text"].tolist(), max_features=max_features)
    x_train = vectorize_texts(train_df["text"].tolist(), vocab)
    y_train = np.array([labels.index(label) for label in train_df["intent"]],
                        dtype=np.int64)
    y_train_oh = one_hot(y_train, len(labels))
    input_size = max(1, x_train.shape[1])
    w1 = rng.normal(0, np.sqrt(2.0 / input_size), size=(input_size, hidden_size)
    ).astype(np.float32)
    ...
```

```

np.savez_compressed(
    LOCAL_MODEL_PATH,
    w1=w1, b1=b1, w2=w2, b2=b2, w3=w3, b3=b3,
    vocab=np.array(json.dumps(vocab, ensure_ascii=False)),
    labels=np.array(json.dumps(labels, ensure_ascii=False)),
)
return {
    "model_path": str(LOCAL_MODEL_PATH),
    "model_type": "numpy_mlp_text_classifier",
    "architecture": {
        "input_features": int(len(vocab)),
        "hidden_layers": [int(hidden_size), int(hidden_size_2)],
        "activation": "relu",
        "output": "softmax",
    },
}

```

3.6.3. Quy trình huấn luyện và metadata

Script huấn luyện không chỉ tạo mô hình mà còn sinh toàn bộ metadata để service runtime đọc lại khi khởi động.

Listing 9: Sinh metadata và dữ liệu fine-tune

```

metadata = {
    "provider": "local_numpy",
    "training_type": "supervised_neural_intent_classification",
    "dataset_path": str(dataset_path),
    "row_count": int(len(df)),
    "train_rows": int(len(train_df)),
    "validation_rows": int(len(valid_df)),
    "intents": sorted(df["intent"].unique().tolist()),
    "local_model": local_model,
    "base_model": args.base_model,
    "train_jsonl": str(TRAIN_JSONL_PATH),
    "validation_jsonl": str(VALID_JSONL_PATH),
    "job": None,
}

```

3.7. Thành phần 2 — Knowledge Graph với Neo4j

3.7.1. Lý do chọn Graph Database

Neo4j phù hợp với bài toán e-commerce vì dữ liệu không chỉ là bảng đơn lẻ mà còn có quan hệ chéo giữa sản phẩm, khách hàng, đơn hàng và tài liệu mô tả. Điều này giúp AI Service

trả lời tốt hơn các truy vấn kiểu:

- khách hàng nào đã đặt đơn nào;
- đơn nào chứa sản phẩm nào;
- sản phẩm nào thuộc danh mục hoặc thương hiệu nào.

3.7.2. Thiết kế Graph Schema

- Node: Product, Customer, Order, Document, Category, Brand.
- Quan hệ: BELONGS_TO, MADE_BY, PLACED, CONTAINS, ABOUT.

Listing 10: Tạo constraint và vector index trong Neo4j

```
def _ensure_indexes(driver) -> None:
    driver.execute_query("""
        CREATE CONSTRAINT document_doc_id IF NOT EXISTS
        FOR (d:Document) REQUIRE d.doc_id IS UNIQUE
    """, database_=NEO4J_DATABASE)
    driver.execute_query(f"""
        CREATE VECTOR INDEX {NEO4J_VECTOR_INDEX} IF NOT EXISTS
        FOR (d:Document) ON d.embedding
        OPTIONS {{indexConfig: {{
            'vector.dimensions': {EMBEDDING_DIMENSIONS},
            'vector.similarity_function': 'cosine'
        }}}
    """, database_=NEO4J_DATABASE)
```

Listing 11: Upsert dữ liệu sản phẩm, đơn hàng và tài liệu vào Neo4j

```
def _upsert_product(driver, payload: Dict) -> None:
    driver.execute_query(
        """
        MERGE (p:Product {id: $id})
        SET p.name = $name,
            p.sku = $sku,
            p.category = $category,
            p.brand = $brand,
            p.price = $price,
            p.discount_price = $discount_price,
            p.stock_quantity = $stock_quantity,
            p.rating = $rating,
            p.payload_json = $payload_json
        WITH p
        MERGE (c:Category {name: coalesce($category, 'Unknown')})
```

```

MERGE (b:Brand {name: coalesce($brand, 'Unknown')})
MERGE (p)-[:BELONGS_TO]->(c)
MERGE (p)-[:MADE_BY]->(b)
"",
id=str(payload.get("id", "")),
name=payload.get("name"),
sku=payload.get("sku"),
category=payload.get("category"),
brand=payload.get("brand"),
price=payload.get("price"),
discount_price=payload.get("discount_price"),
stock_quantity=payload.get("stock_quantity"),
rating=payload.get("rating"),
payload_json=_safe_json(payload),
database_=NEO4J_DATABASE,
)

```

3.8. Thành phần 3 — RAG (Retrieval-Augmented Generation)

3.8.1. Tại sao cần RAG?

RAG giúp AI trả lời dựa trên dữ liệu thật của hệ thống thay vì chỉ dựa vào suy đoán của mô hình. Với e-commerce, đây là phần quan trọng nhất để tra cứu sản phẩm, khách hàng và đơn hàng một cách có căn cứ.

3.8.2. Kiến trúc Multi-Retrieval

Repo hiện tại triển khai một lớp retrieval đa nguồn:

- ưu tiên truy vấn chính xác theo entity ID;
- nếu không có, dùng Neo4j vector index;
- nếu Neo4j chưa sẵn sàng, quay về TF-IDF local.

Listing 12: Cấu trúc RetrievalDoc và hàm tìm kiếm

```

@dataclass
class RetrievalDoc:
    entity_type: str
    entity_id: str
    title: str
    text: str
    payload: Dict[str, Any]
    score: float = 0.0

```

```
def search_retrieval_docs(query: str, top_k: int = 3, scope: Optional[str] =
    None) -> List[RetrievalDoc]:
    normalized_query = (query or "").strip()
    if not normalized_query:
        return []
    if is_neo4j_configured():
        neo4j_docs = _search_neo4j_docs(normalized_query, top_k=top_k, scope=
scope)
        if neo4j_docs:
            return neo4j_docs
    ranked_docs = _rank_docs(normalized_query, build_retrieval_docs(query=
normalized_query, scope=scope))
    return ranked_docs[: max(1, top_k)]
```

Listing 13: Xây dựng tài liệu truy xuất cho product, order và user

```
def _build_product_doc(product: Dict[str, Any]) -> RetrievalDoc:
    searchable_text = " | ".join([
        _normalize(product.get("name")),
        _normalize(product.get("sku")),
        _normalize(product.get("category")),
        _normalize(product.get("brand")),
    ])
    title = f"{product.get('name', 'Product')} ({product.get('sku', '')})"
    return RetrievalDoc("product", str(product.get("id", "")), title,
searchable_text, product)

def _build_order_doc(order: Dict[str, Any]) -> RetrievalDoc:
    searchable_text = " | ".join([
        _normalize(order.get("id")),
        _normalize(order.get("customer_name")),
        _normalize(order.get("customer_email")),
        _normalize(order.get("status")),
    ])
    title = f"Order #{order.get('id', '')} - {order.get('customer_name', '
Customer')}"
    return RetrievalDoc("order", str(order.get("id", "")), title,
searchable_text, order)
```

3.8.3. LLM Router Agent

AI Service có thể dùng OpenAI hoặc Gemini nếu được cấu hình biến môi trường. Nếu không có provider, hệ thống vẫn chạy nhờ phản hồi fallback nội bộ. Đây là điểm giúp demo luôn ổn định ngay cả khi chưa có API key.

3.9. Kết hợp Hybrid — Tích hợp ba thành phần

Luồng hybrid của AI Service là:

1. Nhận message từ client.
2. Resolve intent bằng mô hình local hoặc rule-based fallback.
3. Nếu cần, truy xuất dữ liệu từ retrieval layer và Neo4j.
4. Gọi LLM provider để sinh câu trả lời có cấu trúc JSON.
5. Trả lại response cùng danh sách sources.

Listing 14: Resolve intent và xử lý endpoint chat

```
def _resolve_intent(message: str) -> tuple[Optional[str], float, str]:
    neural_intent, neural_confidence = _predict_neural_intent(message)
    rule_based_intent = _detect_rule_based_intent(message)

    if neural_intent and neural_confidence >= 0.35:
        return neural_intent, neural_confidence, "local_neural_mlp"
    if rule_based_intent:
        return rule_based_intent, 0.25, "rule_based"
    if neural_intent:
        return neural_intent, neural_confidence, "
local_neural_mlp_low_confidence"
    return None, 0.0, "none"

@router.post("/chat", response_model=ChatResponse)
def handle_chat(request: ChatRequest):
    message = (request.message or "").strip()
    resolved_intent, intent_confidence, intent_source = _resolve_intent(message)
    rag_scope = _scope_for_intent(resolved_intent, message)
    should_use_rag = resolved_intent in ("product_inquiry", "order_status", "
customer_lookup") or _is_rag_query(message)
    docs = search_retrieval_docs(message, top_k=3, scope=rag_scope) if
should_use_rag else []
    response = _llm_chat(message, docs, resolved_intent or ("knowledge_lookup"
if docs else None))
    response.sources.append({
        "entity_type": "intent_classifier",
        "entity_id": intent_source,
        "title": resolved_intent or "unknown",
        "score": round(intent_confidence, 4),
    })
    return response
```

Listing 15: Endpoint search và trend trong AI Service

```
@router.post("/search", response_model=SearchResponse)
def search_knowledge_base(request: SearchRequest):
    docs = search_retrieval_docs(request.query, top_k=request.top_k)
    results = [
        SearchHit(
            entity_type=doc.entity_type,
            entity_id=doc.entity_id,
            title=doc.title,
            score=doc.score,
            payload=doc.payload,
        )
        for doc in docs
    ]
    return SearchResponse(query=request.query, scope=detect_scope(request.query), results=results)

@router.post("/trends", response_model=TrendResponse)
def predict_trend(request: TrendRequest):
    recent_months = request.historical_sales[-3:]
    avg_recent = sum(recent_months) / len(recent_months)
    overall_avg = sum(request.historical_sales) / len(request.historical_sales)
    prediction = avg_recent * 1.05
    if prediction > overall_avg:
        trend = "UPWARD"
    elif prediction < overall_avg:
        trend = "DOWNWARD"
    else:
        trend = "STABLE"
    return TrendResponse(predicted_sales=round(prediction, 2), trend=trend)
```

3.10. Đánh giá & Triển khai

Để đáp ứng phần đánh giá và triển khai trong requirement.md, AI Service cần được nhìn theo ba mức chất lượng đầu ra:

- **Không có RAG:** khi câu hỏi chỉ là chào hỏi, tạm biệt hoặc phản hồi cảm xúc, service trả lời trực tiếp bằng intent response.
- **RAG cục bộ:** khi query gắn với mã đơn, SKU, email hoặc tên khách hàng, retrieval sẽ lấy đúng entity từ Neo4j hoặc TF-IDF.
- **LLM có kiểm soát:** khi LLM provider được bật, prompt chỉ nhận context đã được thu gọn và buộc trả JSON có cấu trúc.

Kiểu đầu vào	Đường xử lý	Đặc trưng đầu ra
Greeting / farewell	Rule-based fallback local	+ Câu trả lời ngắn, ổn định, không cần retrieval.
Product / order / user lookup	Intent model Neo4j/TFIDF LLM	+ Trả lời có nguồn, có ngữ cảnh, tránh bịa dữ liệu.
Trend request	Lực tính toán số học đơn giản	Trả về xu hướng UPWARD/DOWNWARD/STABLE.

Quy trình triển khai AI Service:

1. Cấu hình biến môi trường cho AI_PROVIDER, OPENAI_API_KEY hoặc GEMINI_API_KEY, NEO4J_URI.
2. Chạy script đồng bộ dữ liệu sang Neo4j bằng `ingest_to_neo4j.py`.
3. Khởi động AI service cùng các service nghiệp vụ bằng Docker Compose.
4. Mở frontend, nhập câu hỏi từ widget chat và kiểm tra phản hồi.
5. Dùng endpoint `/api/ai/search` và `/api/ai/trends` để kiểm thử riêng từng nhánh.

3.11. Tích hợp hệ thống Chat với dữ liệu

Phần tích hợp chat với dữ liệu được thể hiện rõ trong cả backend và frontend:

- Backend: `main.py` gọi `search_retrieval_docs()` để lấy product, order hoặc customer có liên quan.
- Retrieval: `retrieval.py` xây dựng văn bản ngữ nghĩa từ dữ liệu thực của hệ thống, không dùng câu trả lời sinh tự do từ đầu.
- Frontend: `chatbot.tsx` gửi message đến `/api/backend/ai/chat` và render trực tiếp text trả về.

Listing 16: Khung chat nổi trên frontend được gắn toàn cục

```
import Chatbot from "@components/chat/chatbot";

export default function RootLayout({ children }) {
  return (
    <html lang="vi">
      <body>
        {children}
        <Chatbot />
      </body>
    </html>
  );
}
```

```
</html>
);
}
```

3.12. Tích hợp AI vào hệ thống thương mại điện tử

AI được tích hợp vào e-commerce không chỉ ở mức trả lời câu hỏi, mà còn ở mức hỗ trợ vận hành:

- trong dashboard, admin/staff có thể tra cứu user, product, order và payment;
- trong customer order portal, người dùng chọn hàng, tạo đơn và giữ được ngữ cảnh đặt hàng;
- chatbot tư vấn được gắn ở mọi trang, nên người dùng có thể hỏi ngay khi đang xem dashboard hoặc thao tác đơn hàng.

Frontend của hệ thống đang đặt các màn hình chính theo vai trò:

- / - dashboard điều hành;
- /customer/login - đăng nhập demo theo email;
- /customer/order - tạo đơn hàng;
- /staff/products và /admin/products - quản lý sản phẩm;
- chatbot AI - widget nổi ở góc dưới phải, dùng như trợ lý tư vấn xuyên suốt.

3.13. Triển khai AI Service vào hệ thống Microservices

AI Service được triển khai như một service độc lập trong hệ sinh thái microservices:

- chạy trên port riêng;
- đọc dữ liệu từ user/product/order service thông qua HTTP;
- có CORS cấu hình rộng để phục vụ frontend demo;
- có healthcheck trả về trạng thái provider, local model, Neo4j và metadata huấn luyện.

Listing 17: Healthcheck của AI Service

```
@router.get("/health")
def healthcheck():
    return {
        "status": "ok",
        "service": "ai_service",
        "ai_provider": AI_PROVIDER,
        "active_provider": _active_provider(),
        "neo4j_configured": is_neo4j_configured(),
        "local_intent_model_configured": bool(intent_model),
```

```
"training_metadata": training_metadata or None,
}
```

3.14. Checklist đánh giá Chương 3

Hạng mục	Trạng thái
Intent classification cục bộ	Đã hiện thực bằng NumPy MLP.
Retrieval trên dữ liệu nghiệp vụ	Đã hiện thực qua Neo4j và TF-IDF fallback.
Knowledge graph	Đã hiện thực với Product, Customer, Order, Document.
LLM provider fallback	Đã hỗ trợ OpenAI, Gemini và local fallback.
Endpoint chat/search/trends	Đã hiện thực trong FastAPI.
Healthcheck và metadata	Đã hiện thực và trả về trạng thái runtime.

3.15. Kết luận Chương 3

Chương 3 cho thấy AI Service không phải là một chatbot rời rạc, mà là một service có đủ dữ liệu, retrieval, graph và lớp sinh phản hồi. Intent model là NumPy MLP, RAG dùng Neo4j vector index và TF-IDF fallback, còn AI runtime chạy qua các endpoint `/chat`, `/search`, `/trends` và `/health`.

CHƯƠNG 4: XÂY DỰNG HỆ THỐNG HOÀN CHỈNH

4.1. Kiến trúc tổng thể hệ thống

4.1.1. Tổng quan

Phần vận hành của hệ thống xoay quanh ba lớp chính: frontend Next.js, gateway trung gian và các service nghiệp vụ Django/FastAPI. Cart và Shipping chưa tách thành service độc lập; thay vào đó, các vai trò này được gom vào `order_service` và `payment_service`. Dù vậy, cấu trúc tổng thể vẫn đi đúng tinh thần Microservices với tách biệt service, database riêng và giao tiếp qua HTTP.

Thành phần	Cổng / vị trí	Trách nhiệm
Frontend client	3000	Dashboard, đăng nhập demo, đặt hàng và widget chat AI.
Gateway	8000	Nhận request từ client và chuyển tiếp tới service đích.
User service	8001	Quản lý user profile, role, trạng thái và summary.
Product service	8002	Quản lý catalog sản phẩm, danh mục, rating và tồn kho.
Order service	8003	Tạo đơn hàng, validate user/product và lưu snapshot item.
Payment service	8004	Tạo payment từ order và lưu trạng thái giao dịch.
AI service	8005	Chat, search, trends, retrieval và healthcheck.
PostgreSQL	5433 -> 5432	Lưu dữ liệu quan hệ của các service nghiệp vụ.
Neo4j	7474/7687	Lưu graph, document embedding và vector index.

4.1.2. Cấu trúc thư mục dự án

Code backend nằm trong `e-commerce-server`, frontend nằm trong `e-commerce-client`, còn file orchestration ở gốc dự án là `docker-compose.yaml`. Cách bố trí này làm cho từng service có thể build riêng nhưng vẫn được ghép lại thành một hệ thống thống nhất khi chạy demo.

4.2. API Gateway — Nginx

4.2.1. Vai trò và trách nhiệm

Vai trò gateway được hiện thực bằng hai lớp: một route proxy trong Next.js ở phía client và một gateway Django ở backend. Cấp lớp này đảm nhiệm việc chuẩn hoá đường dẫn, giảm phụ thuộc trực tiếp từ giao diện tới từng service, và gom lỗi upstream về một format để xử lý hơn.

Listing 18: Next.js proxy route chuyển request sang backend

```
const backendBaseUrl =
  process.env.ECOMMERCE_API_BASE_URL?.replace(/\/$/, "") ?? "http
  ://127.0.0.1:8000";
```

```

async function proxy(request: NextRequest, params: { path: string[] }) {
  const backendPath = normalizeBackendPath(request.nextUrl.pathname);
  const search = request.nextUrl.search;
  const target = `${backendBaseUrl}/api/${backendPath}${search}`;
  const response = await fetch(target, {
    method: request.method,
    headers: {
      "Content-Type": request.headers.get("content-type") ?? "application/json",
    },
    body: request.method === "GET" ? undefined : await request.text(),
    cache: "no-store",
  });
  ...
}

```

4.2.2. Cấu hình gateway trong frontend

Frontend gọi backend qua helper thống nhất thay vì fetch trực tiếp đến nhiều URL rời rạc. Điều này giúp đổi base URL dễ hơn và giữ luồng call đồng nhất.

Listing 19: Helper gọi backend từ frontend

```

export async function requestBackendJson<T>(path: string, init?: RequestInit):
  Promise<T> {
  const safePath = path.replace(/~/+/ , "");
  const response = await fetch(`/api/backend/${safePath}`, {
    ...init,
    headers: {
      "Content-Type": "application/json",
      ...(init?.headers ?? {}),
    },
  });

  if (!response.ok) {
    const detail = await extractErrorMessage(response);
    const message = detail ? `HTTP ${response.status}: ${detail}` : `HTTP ${
      response.status}: Request failed`;
    throw new Error(message);
  }

  return (await response.json()) as T;
}

```

4.3. Authentication — JWT

4.3.1. Luồng xác thực hiện tại

Hệ thống chưa triển khai JWT đầy đủ. Thay vào đó, đang dùng luồng demo-login theo email để phục vụ đồ án: người dùng nhập email, backend trả về hồ sơ user, frontend lưu session local và định tuyến theo role. Đây là một bước đệm hợp lý cho giai đoạn hoàn thiện JWT sau này.

Listing 20: Luồng login demo theo email ở frontend

```
const response = await requestBackendJson<LoginResponse>("users/login/", {
  method: "POST",
  body: JSON.stringify({ email }),
});

saveSessionUser(response.user);
if (response.user.role === "customer") {
  router.push("/customer/order");
  return;
}
```

4.3.2. Cài đặt JWT trong User Service (hướng mở rộng)

JWT nên được phát hành từ `user_service`, sau đó gateway kiểm tra token trước khi chuyển request tới service đích. Trong hệ thống hiện tại, phần này mới ở mức hướng phát triển tiếp theo, chưa có access token/refresh token thực sự.

- Hiện trạng: login demo, session local, role-based redirect.
- Hướng mở rộng: phát hành access token, refresh token và middleware xác thực ở gateway.
- Giá trị kiến trúc: chuẩn hoá quyền truy cập trước khi đi vào service nghiệp vụ.

4.4. Giao tiếp giữa các Service

4.4.1. Pattern gọi API nội bộ có xử lý lỗi

Từ docker compose đến helper frontend đều cho thấy một mô hình giao tiếp khá rõ ràng: frontend không gọi trực tiếp service con mà đi qua proxy; service nghiệp vụ thì gọi nhau qua URL nội bộ trong mạng Docker. Phần lỗi được gom lại ở helper frontend và response upstream, giúp UI có thể hiển thị thông báo dễ hiểu hơn.

- Frontend -> Next.js proxy -> gateway -> service đích.
- `order_service` gọi `user_service` và `product_service` để validate dữ liệu.
- `payment_service` gọi `order_service` để lấy thông tin đơn trước khi tạo payment.

- `ai_service` đọc dữ liệu từ `user/product/order` và Neo4j để thực hiện retrieval.

Listing 21: Định tuyến nội bộ giữa các service trong docker compose

```
gateway:
  environment:
    USER_SERVICE_URL: http://ecommerce-user-service:8001/api/users/
    PRODUCT_SERVICE_URL: http://ecommerce-product-service:8002/api/products/
    ORDER_SERVICE_URL: http://ecommerce-order-service:8003/api/orders/
    PAYMENT_SERVICE_URL: http://ecommerce-payment-service:8004/api/payments/
    AI_SERVICE_URL: http://ecommerce-ai-service:8005/api/ai/

order_service:
  environment:
    USER_SERVICE_URL: http://ecommerce-user-service:8001/api/users/
    PRODUCT_SERVICE_URL: http://ecommerce-product-service:8002/api/products/
```

4.5. Luồng mua hàng End-to-End — Code hoàn chỉnh

4.5.1. *Luồng nghiệp vụ ở frontend*

Người dùng customer đăng nhập theo email, chuyển sang trang đặt hàng, chọn sản phẩm, gửi order lên backend, rồi tạo payment cho order vừa sinh. Phí ship được tính ngay trong frontend preview và được đẩy xuống backend như một phần của payload order.

Listing 22: Tạo đơn hàng từ giao diện customer order

```
const order = await requestBackendJson<Order>("orders/", {
  method: "POST",
  body: JSON.stringify({
    user_id: sessionId,
    shipping_address: shippingAddress,
    shipping_city: shippingCity,
    notes,
    items: items.map((item) => ({
      product_id: item.productId,
      quantity: item.quantity,
    })),
  }),
});
```

Listing 23: Dashboard tạo payment từ order

```
const payment = await requestBackendJson<{ id: number; order_id: number }>("
  payments/", {
  method: "POST",
```

```
body: JSON.stringify({
  order_id: paymentForm.orderId,
  method: paymentForm.method,
  source: paymentForm.source,
}),
});
```

4.5.2. Xử lý trong Order Service và Payment Service

Order Service là nơi gom logic validate user/product, tính subtotal và snapshot item. Payment Service lấy order đã tồn tại, tạo payment và ghi lại trạng thái giao dịch demo. Vì hệ thống chưa tách Shipping Service riêng, nên `shipping_fee` và địa chỉ giao hàng được gộp vào Order.

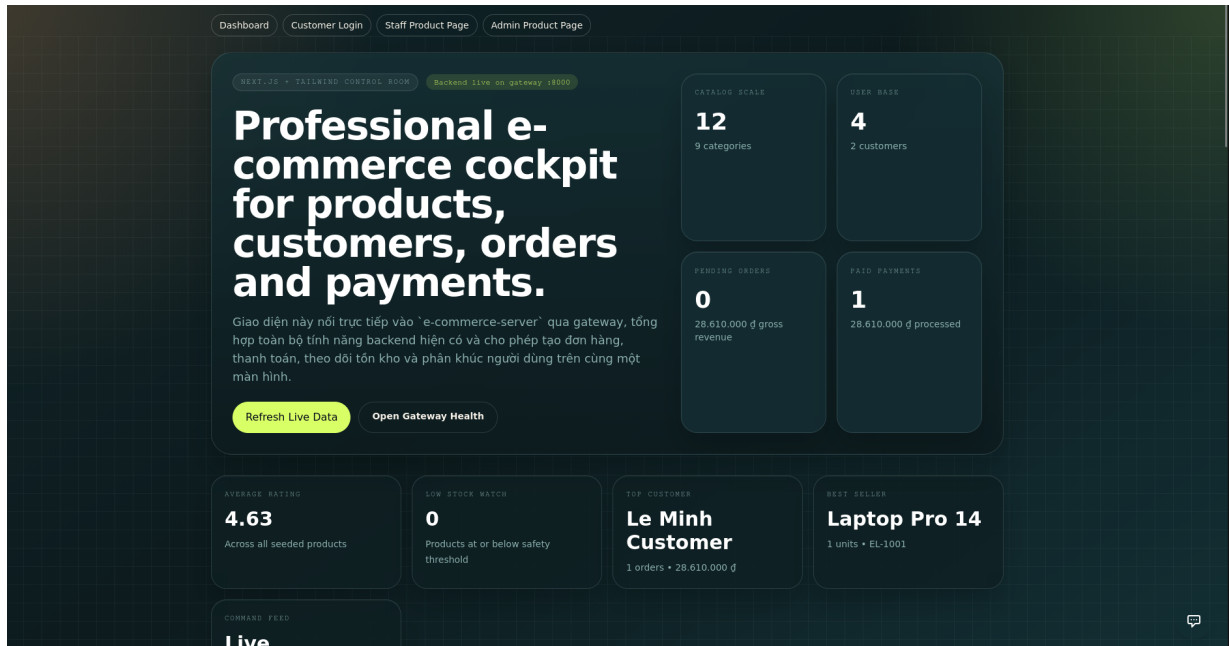
Listing 24: Logic tạo order và payment rút gọn theo code hiện tại

```
user = _fetch_user(payload["user_id"])
for item in items_payload:
    product = _fetch_product(item["product_id"])
    quantity = int(item["quantity"])
    unit_price = Decimal(str(product["current_price"]))
    subtotal += unit_price * quantity

order = Order.objects.create(
    user_id=user["id"],
    customer_name=user["full_name"],
    customer_email=user["email"],
    shipping_address=payload.get("shipping_address") or user.get("address") or "
Demo address",
    shipping_city=payload.get("shipping_city") or user.get("city") or "Ho Chi
Minh City",
    subtotal_amount=subtotal,
    shipping_fee=shipping_fee,
    total_amount=subtotal + shipping_fee,
)
```

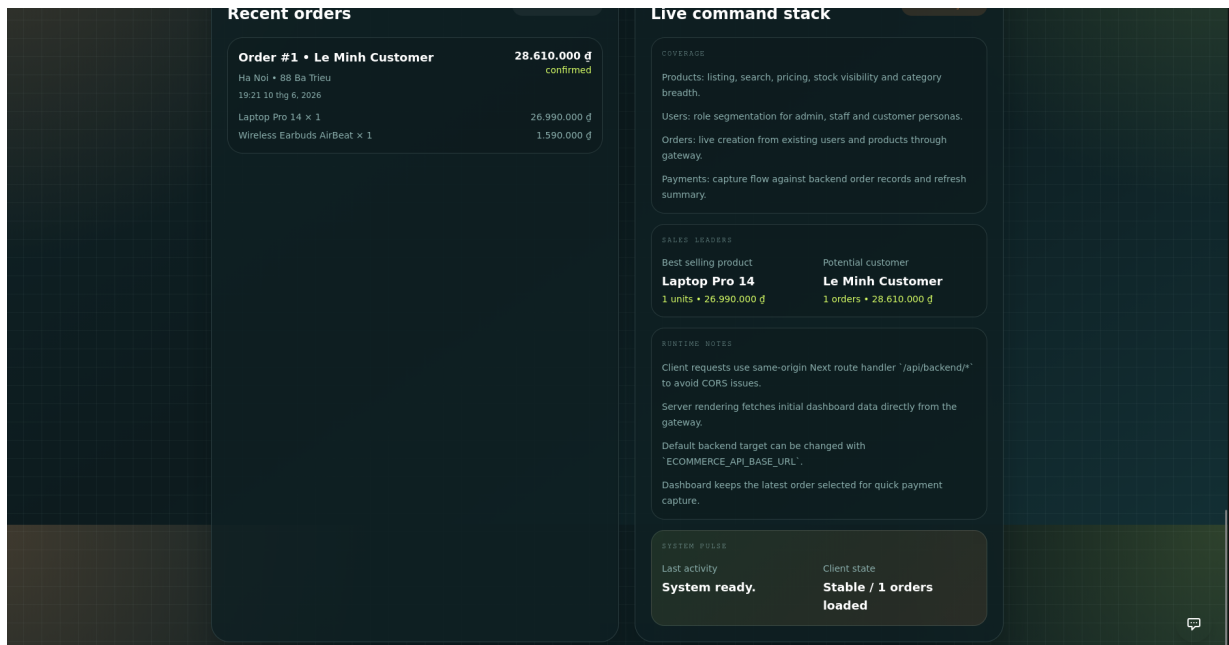
4.6. Minh họa giao diện frontend

Bộ ảnh dưới đây mô tả các màn hình chính của frontend Next.js trong quá trình demo hệ thống. Nhóm ảnh này cho thấy cách client đóng vai trò lớp trình bày trung tâm: dashboard tổng quan, màn hình đăng nhập theo email, trang đặt hàng, màn hình quản lý sản phẩm cho staff/admin và widget trợ lý AI gần toàn cục. Các ảnh cũng phản ánh cách frontend kết nối qua gateway để lấy dữ liệu thật từ backend thay vì chỉ là giao diện tĩnh.



Hình 7: Dashboard tổng quan của hệ thống frontend

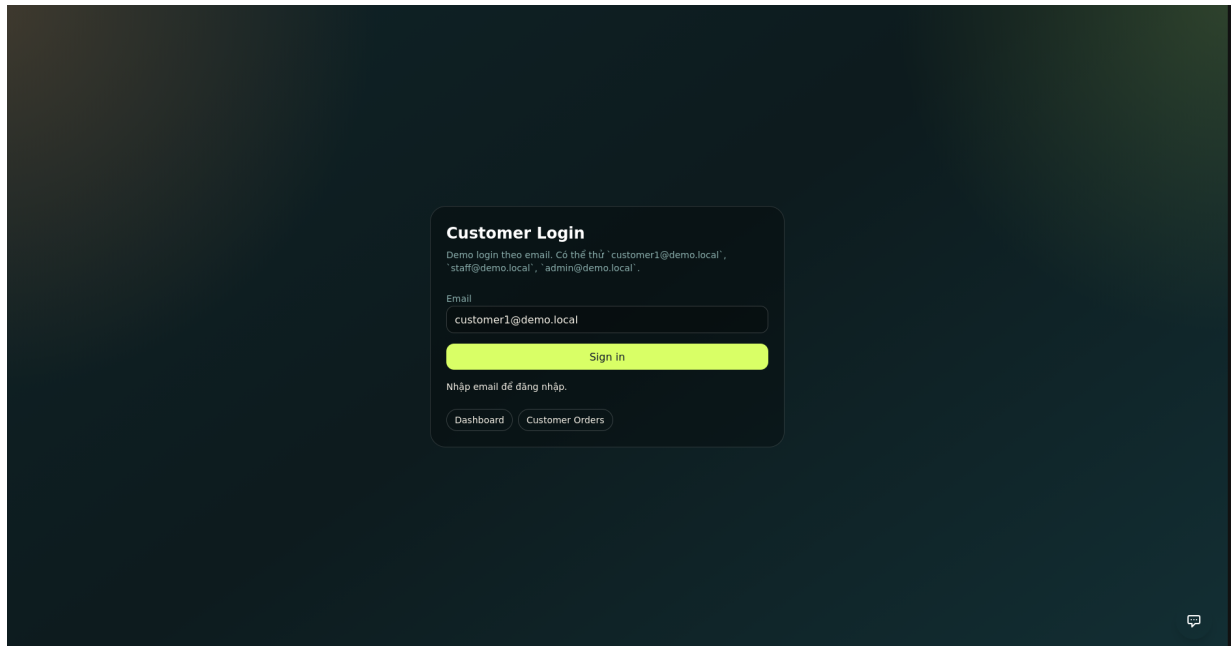
Dashboard là màn hình trung tâm của hệ thống, nơi tập hợp các chỉ số vận hành như số sản phẩm, số người dùng, đơn hàng chờ xử lý, payment đã ghi nhận, customer đứng đầu và sản phẩm bán chạy nhất. Màn hình này thể hiện rõ vai trò của lớp client trong việc tổng hợp dữ liệu từ nhiều service thành một góc nhìn điều hành duy nhất.



Hình 8: Dashboard chi tiết với luồng hoạt động và trạng thái hệ thống

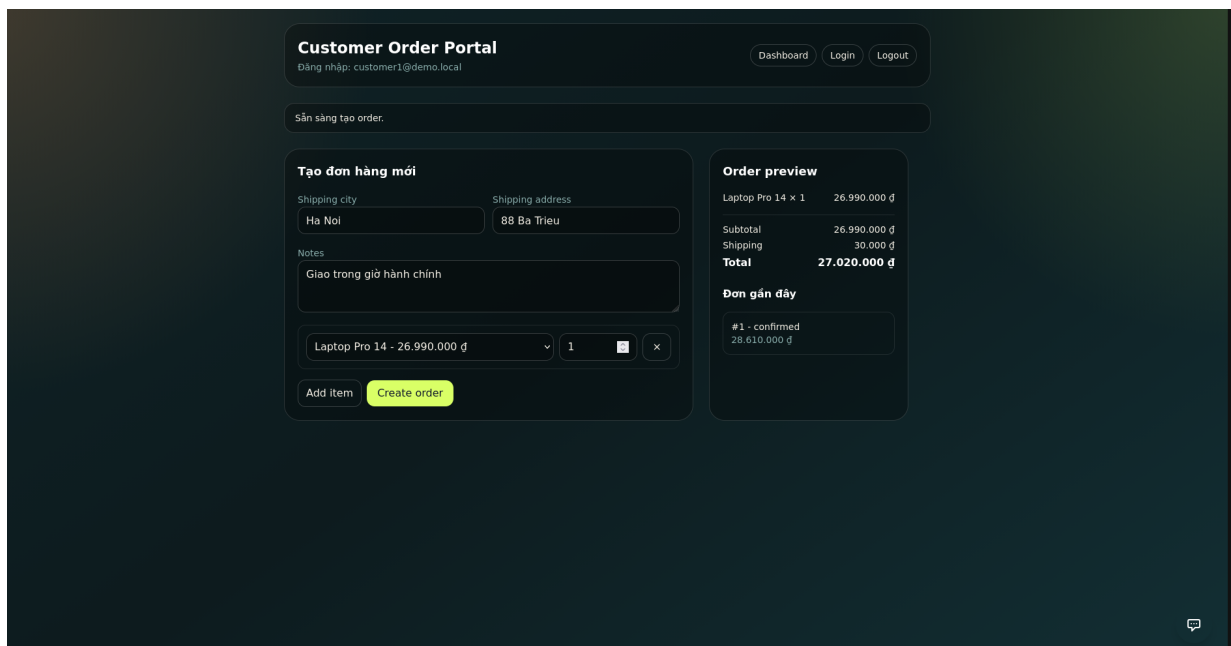
Ở ảnh chi tiết của dashboard, phần recent orders, live command stack và runtime notes cho thấy frontend không chỉ hiển thị số liệu, mà còn mô tả luôn trạng thái luồng nghiệp vụ

đang chạy. Điều này giúp người dùng dễ theo dõi order gần nhất, trạng thái client, và cách các thao tác phía frontend đang tương tác với backend.



Hình 9: Màn hình đăng nhập customer theo email

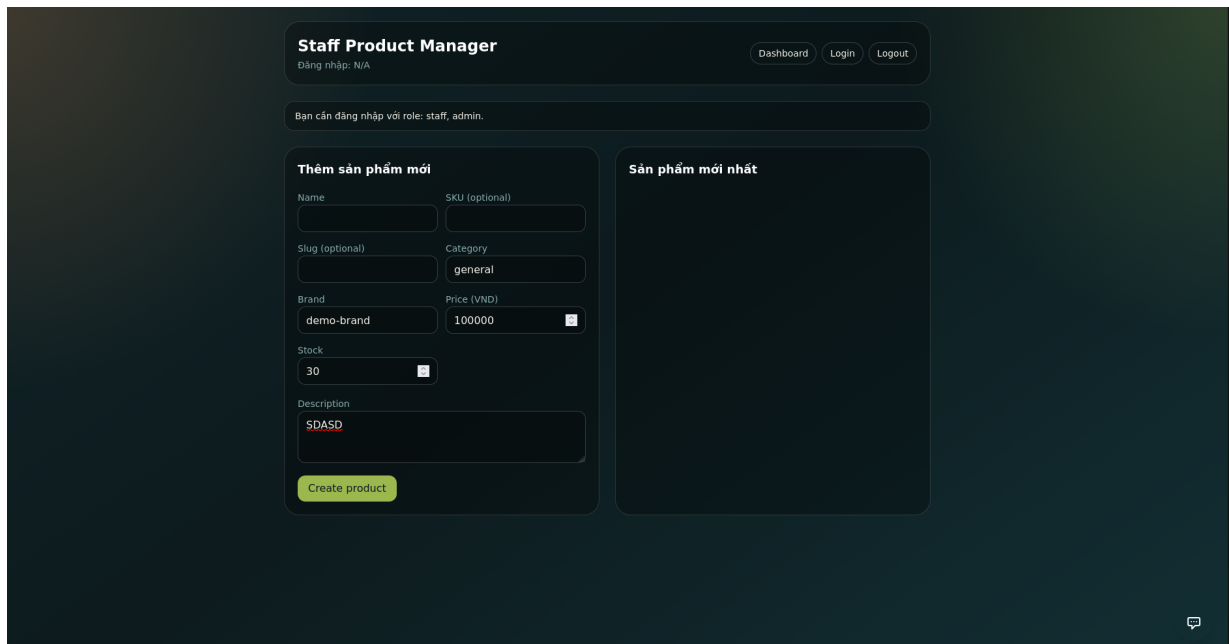
Màn hình đăng nhập customer thể hiện cơ chế xác thực demo của hệ thống. Người dùng chỉ cần nhập email là có thể đi vào luồng khách hàng, từ đó mở rộng sang trang đặt hàng hoặc dashboard. Cách làm này phù hợp với giai đoạn hiện tại vì tách riêng luồng xác thực khỏi phần xử lý nghiệp vụ, đồng thời giữ trải nghiệm demo đơn giản, nhanh và ổn định.



Hình 10: Customer Order Portal để tạo đơn hàng

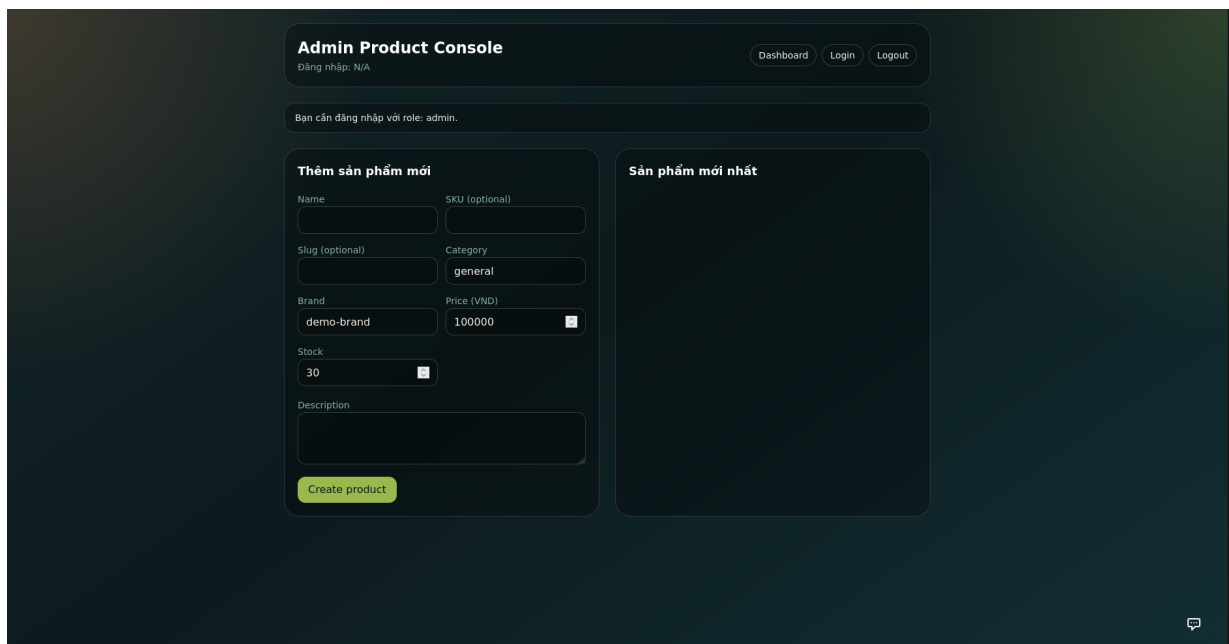
Trang Customer Order Portal là nơi khách hàng chọn sản phẩm, nhập địa chỉ giao hàng,

ghi chú đơn và xem trước tổng tiền. Cấu trúc giao diện chia thành khu vực tạo order bên trái và phần order preview bên phải, giúp người dùng nhìn ngay được sự thay đổi của subtotal, phí ship và tổng tiền trước khi gửi request xuống backend.



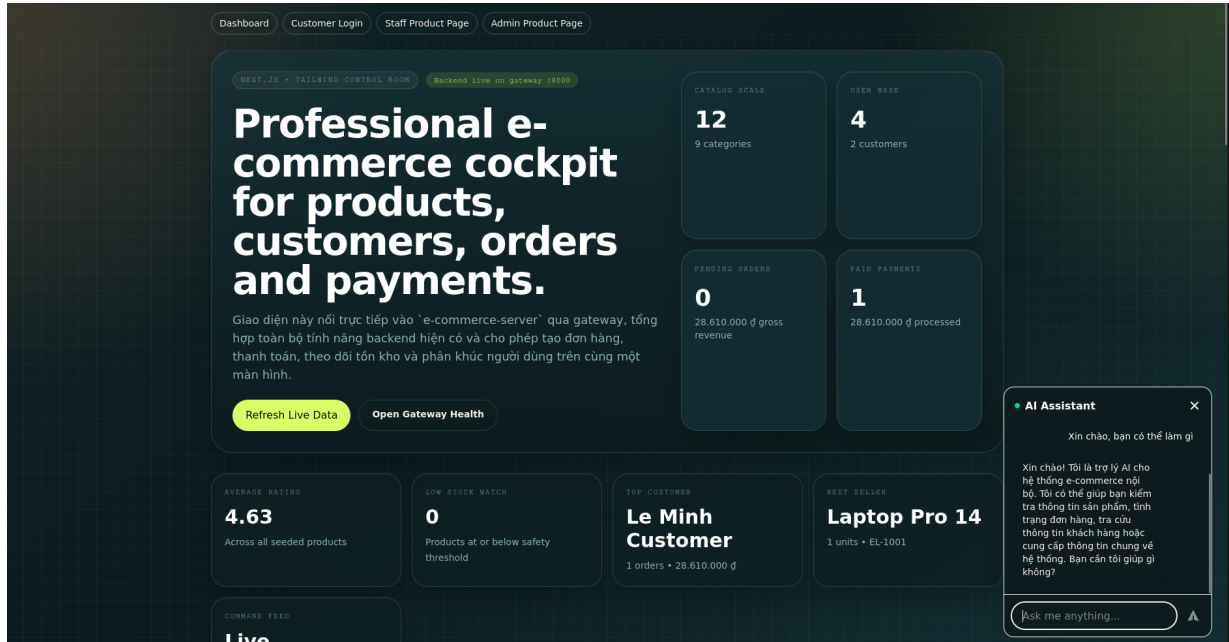
Hình 11: Giao diện Staff Product Manager

Màn hình Staff Product Manager dành cho người dùng có vai trò staff. Ở đây, frontend cung cấp form tạo sản phẩm mới cùng một vùng hiển thị sản phẩm mới nhất. Giao diện này phản ánh đúng phân quyền nghiệp vụ: staff được phép hỗ trợ quản lý catalog nhưng không cần đi sâu vào phần vận hành của admin.



Hình 12: Giao diện Admin Product Console

Admin Product Console có bố cục tương tự trang của staff nhưng nhấn mạnh quyền quản trị cao hơn. Ảnh này cho thấy hệ thống đã tách rõ các màn hình theo vai trò, trong đó admin có thể thao tác với catalog ở mức quản lý trung tâm, hỗ trợ cho mục tiêu vận hành và kiểm soát dữ liệu toàn cục.



Hình 13: AI Assistant widget gắn toàn cục trên frontend

Widget AI Assistant được gắn nổi ở góc màn hình và xuất hiện xuyên suốt trong giao diện. Nó thể hiện lớp hỗ trợ hội thoại tích hợp sẵn, cho phép người dùng đặt câu hỏi ngay khi đang xem dashboard hoặc thao tác đơn hàng. Về mặt trải nghiệm, đây là điểm nối giữa frontend và AI service, giúp hệ thống có cảm giác liền mạch hơn thay vì tách rời thành các màn hình độc lập.

4.7. Docker hoá toàn bộ hệ thống

4.7.1. Dockerfile chuẩn cho Django Services

Mỗi service Django đều chạy cùng một pattern: build image riêng, mount code vào container, dùng `entrypoint.sh` để chờ DB, migrate và seed dữ liệu. Cách này làm cho các service đồng nhất về quy trình khởi động, giảm tình trạng service lên trước database.

4.7.2. docker-compose.yml hoàn chỉnh

File `docker-compose.yml` ở gốc dự án ghép toàn bộ hệ thống lại thành một stack chạy được ngay trên máy local. Service hiện có trong repo gồm gateway, client, user, product, order, payment, ai, PostgreSQL và Neo4j.

Listing 25: Trích cấu hình docker compose thực tế của repo

```

services:
  postgres:
    image: postgres:16-alpine
    ports:
      - "${DB_EXPOSE_PORT:-5433}:5432"

  neo4j:
    image: neo4j:5.26
    ports:
      - "${NEO4J_HTTP_PORT:-7474}:7474"
      - "${NEO4J_BOLT_PORT:-7687}:7687"

  gateway:
    build:
      context: ./e-commerce-server/gateway
    ports:
      - "${GATEWAY_PORT:-8000}:8000"
    depends_on:
      - user_service
      - product_service
      - order_service
      - payment_service
      - ai_service

```

4.8. Demo kết quả — Minh hoạ API calls

4.8.1. Khởi động hệ thống

Trong repo hiện tại, một lệnh `docker compose up -build` sẽ dựng toàn bộ stack. Nếu muốn theo dõi trạng thái container, có thể dùng `docker compose ps` hoặc `docker compose logs`.

Listing 26: Khởi động toàn bộ hệ thống

```

cd e-commerce-microservice
docker compose up --build

```

4.8.2. Demo luồng mua hàng bằng curl

Luồng kiểm thử phù hợp với code hiện tại gồm: healthcheck, lấy summary, lấy danh sách sản phẩm, tạo order, tạo payment và kiểm tra AI chat.

Listing 27: Một số lệnh kiểm thử API

```

curl http://localhost:8000/health/
curl http://localhost:8000/api/users/summary/

```

```
curl http://localhost:8000/api/products/summary/
curl http://localhost:8000/api/orders/summary/
curl http://localhost:8000/api/payments/summary/
curl -X POST http://localhost:8000/api/ai/chat -H "Content-Type: application/json" -d '{"message": "where is my order"}'
```

4.9. Logging và Monitoring

4.9.1. Cấu hình Logging tập trung (Django)

Hiện tại logging chủ yếu đi qua stdout/stderr của container, nên cách quan sát thực tế là dùng `docker compose logs`. Cách làm này phù hợp cho đồ án, nhưng nếu đưa lên production thì nên đẩy log về hệ thống tập trung để dễ truy vết xuyên service.

4.9.2. Monitoring với Prometheus + Grafana

Hệ thống chưa triển khai đầy đủ Prometheus/Grafana, nhưng về mặt kiến trúc đây là lớp nên bổ sung tiếp theo để hoàn thiện hơn. Các metric đáng theo dõi là response time, error rate, throughput và số lượng request theo service.

- Hiện trạng: chưa có scrape target và dashboard metrics thực sự trong repo.
- Hướng mở rộng: thêm Prometheus, Grafana và metric endpoint cho từng service.
- Lợi ích: nhìn thấy bottleneck thật ở gateway, order hoặc AI service.

4.10. Đánh giá hệ thống

4.10.1. Hiệu năng

Hệ thống chưa có script benchmark chính thức, nên phần đánh giá hiệu năng ở đây được hiểu là đánh giá kiến trúc và luồng xử lý. Dựa trên pipeline hiện tại, các endpoint summary và list dữ liệu sẽ nhẹ hơn nhiều so với luồng tạo order hay chat AI, vì hai luồng sau phải đi qua validate liên service và/hoặc RAG.

Nhóm endpoint	Mức chi phí	Lý do
Summary/list endpoints	Thấp	Chủ yếu đọc dữ liệu và trả JSON.
Order/payment flow	Trung bình	Có validate user/product và ghi DB.
AI chat/search	Cao nhất	Đi qua retrieval, Neo4j và LLM router.

4.10.2. Khả năng mở rộng

Kiến trúc hiện tại vẫn giữ đúng tinh thần scale theo service. Nếu `product_service` hoặc `ai_service` trở thành bottleneck, ta có thể tăng tài nguyên cho riêng service đó mà không cần kéo cả hệ thống đi cùng. Phần chưa hoàn thiện là Nginx riêng, JWT thực sự và message queue cho các tác vụ bất đồng bộ.

4.10.3. Ưu điểm hệ thống

- Fault isolation rõ hơn monolithic vì mỗi service chạy container riêng.
- Frontend có proxy thống nhất, giảm phụ thuộc vào địa chỉ backend.
- AI service tách riêng nên cập nhật logic tư vấn không kéo theo code nghiệp vụ.
- Dữ liệu quan trọng của order được snapshot, tránh mất lịch sử khi sản phẩm thay đổi.
- Cấu trúc Docker Compose giúp chạy lại demo nhanh và nhất quán.

4.10.4. Nhược điểm và hướng cải thiện

Nhược điểm	Mức độ	Hướng cải thiện
Chưa có JWT thực sự	Cao	Bổ sung token access/refresh và middleware ở gateway.
Chưa có Nginx riêng	Trung bình	Đặt Nginx làm reverse proxy đúng như mô hình tham khảo.
Chưa có message queue	Cao	Thêm RabbitMQ hoặc Redis cho event bất đồng bộ.
Chưa có distributed tracing	Trung bình	Tích hợp Jaeger/OpenTelemetry để debug xuyên service.
Chưa có monitoring đầy đủ	Trung bình	Thêm Prometheus + Grafana cho metrics production.

4.11. Checklist đánh giá Chương 4

Hạng mục	Trạng thái trong repo hiện tại
Sơ đồ kiến trúc tổng thể	Đạt, có đủ client, gateway, service nghiệp vụ, PostgreSQL và Neo4j.

API Gateway config	Đạt ở mức Next.js proxy + Django gateway, chưa có Nginx riêng.
JWT Authentication	Chưa hoàn chỉnh, mới có demo login theo email.
Service communication	Đạt ở mức sync call nội bộ và helper xử lý lỗi.
Flowchart mua hàng	Đạt, có luồng order/payment đầy đủ; Shipping gộp vào order.
Order Service code	Đạt.
Payment Service code	Đạt, payment được tạo từ order hiện có.
Shipping Service code	Mô hình logic / hướng mở rộng.
Dockerfile	Đạt, mỗi service có build riêng và entrypoint.
docker-compose	Đạt, dựng được toàn bộ stack local.
Demo curl	Đạt, có healthcheck, summary và AI chat.
Logging	Đạt ở mức container logs.
Đánh giá hệ thống	Đạt, có nhận xét ưu/nhược điểm và hướng mở rộng.

4.12. Kết luận Chương 4 & Toàn bộ tiểu luận

Chương 4 hoàn thiện vòng tròn kỹ thuật từ kiến trúc đến vận hành thực tế. Dù hệ thống chưa có Nginx riêng, JWT đầy đủ hay monitoring production hoàn chỉnh, nó vẫn chứng minh được tính khả thi của kiến trúc Microservices nhờ gateway rõ ràng, giao tiếp liên service có kiểm soát, Docker Compose orchestration và AI service tách biệt. Nhìn toàn bộ tiểu luận, Chương 1 đặt nền tảng, Chương 2 dựng khung thiết kế, Chương 3 đem lại lớp trí tuệ, còn Chương 4 biến tất cả thành một bản demo vận hành được.

Tài liệu tham khảo

- Martin Fowler, James Lewis, *Microservices: a definition of this new architectural term*.
- Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*.
- Tài liệu chính thức của Django, FastAPI, Next.js, Docker, PostgreSQL và Neo4j.

- Mã nguồn dự án `e-commerce-microservice` trong repository bài làm.